



BSc Thesis
Computer Science

Recommender Systems

Collaborative filtering

Martin Vigholt Nyholm-Andersen

Supervised by Tuukka Ruotsalo

January 2026



Martin Vigholt Nyholm-Andersen

Recommender Systems

BSc Thesis - Computer Science, January 2026

Supervisors: Tuukka Ruotsalo

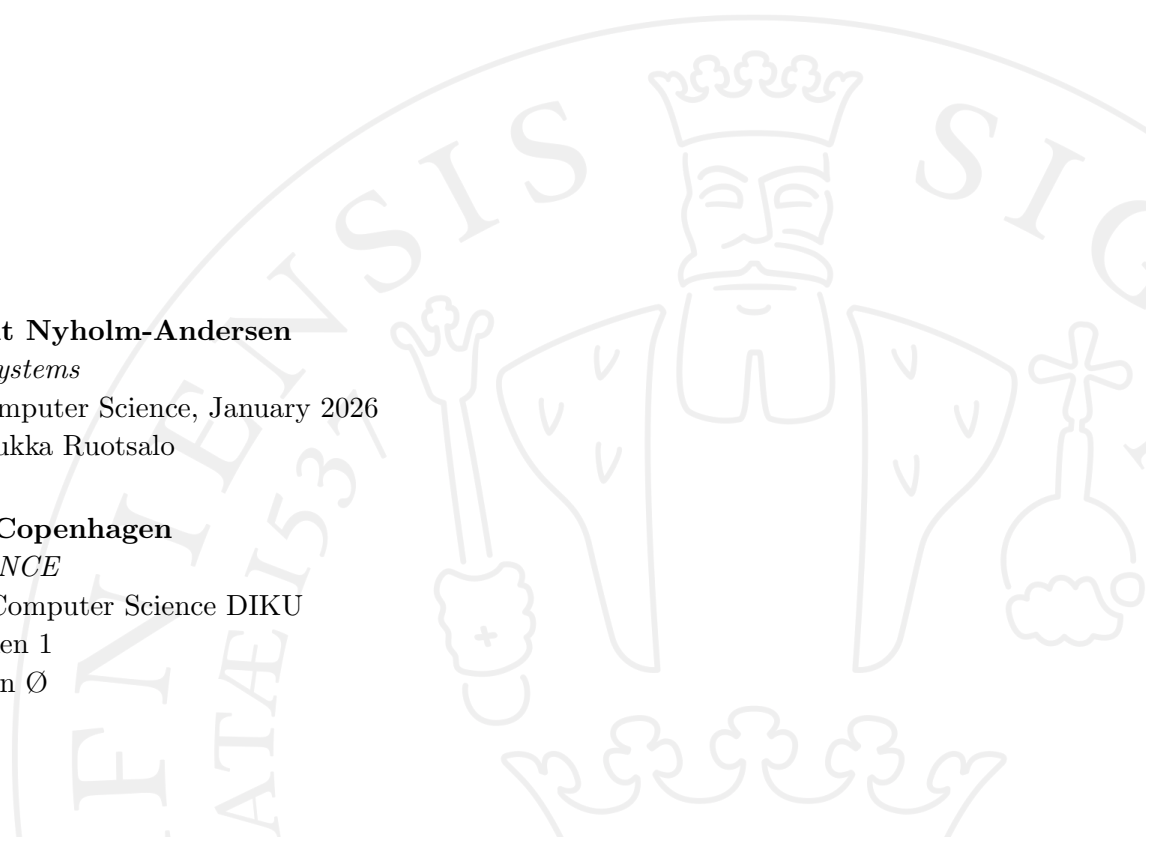
University of Copenhagen

Faculty of SCIENCE

Department of Computer Science DIKU

Universitetsparken 1

2100 Copenhagen Ø



Abstract

Recommender systems play a central role in modern digital platforms by helping users manage information overload. However, choosing the best model for a given task can be challenging.

This thesis presents a comparative study of four collaborative filtering approaches: Item-based K-Nearest Neighbor, Bayesian Personalized Ranking, Neural Matrix Factorization, and Graph Convolutional Matrix Completion. Using the open-source framework RecBole, the models are evaluated under identical experimental conditions on four benchmark datasets of varying size and characteristics. Each dataset is processed using three distinct preprocessing strategies to analyze the impact of explicit and implicit feedback representations. Model performance is assessed with standardized ranking based metrics at cutoff levels $K=10$ and $K=20$, including Precision, Recall, MRR, Hit Rate, and NDCG.

The results demonstrate that no single model consistently outperforms the others across all datasets. Instead, effectiveness, scalability, and robustness vary with data type and preprocessing choices, showing that there is no universally optimal collaborative filtering model and highlighting the need for application specific and resource aware model selection.

Contents

1	Introduction	1
2	Background	3
2.1	History	3
2.2	Modern methods	3
2.3	Evaluation and present challenges	4
3	Methods	5
3.1	ItemKNN	5
3.2	BPR	6
3.3	NeuMF	7
3.4	GCMC	8
4	Experiments	11
4.1	Data	12
4.2	Data processing	12
4.3	Setup	14
5	Results	15
6	Discussion & Conclusion	23
7	References	25
A	Experimental environment	27
A.1	Hardware environment	27
A.2	Software environment	27
B	impR learning curves	28
C	Python scripts	31
C.1	Data processing	31
C.2	Model execution	34
C.3	Log analysis and plotting	36
C.4	Make tables	41
D	Recbole data	48
E	Log examples with hyperparameters	49
E.1	ItemKNN log	49
E.2	BPR log	51
E.3	NeuMF log	53
E.4	GCMC log	56
F	AI-declaration	59

1 Introduction

Recommender systems are tools designed to assist users in navigating the overwhelming amounts of digital information by filtering and predicting items such as products, movies, or news that align with the individual users preferences [4]. A recommender system can be defined as an algorithmic tool that predicts the relevance of unseen items for users based on either a single data type alone, or a combination of multiple data types, such as historical interactions, contextual information, and behavioral patterns [2]. The problem these systems address (often called information overload) appears when users are confronted with a large amount of content and limited ability to evaluate all available options. By personalizing information access, recommender systems serve as a decision support tool that streamline content discovery and aim to improve user satisfaction [4].

Recommender systems have evolved as different types. The earliest and most influential type is collaborative filtering (CF), introduced through the experimental mail filtering system called Tapestry (1992) and the GroupLens news recommender system (1994) [2]. Collaborative filtering relies on patterns of user-item interactions to predict preferences. User-user based CF identifies similar users and recommends items preferred by a user’s peers. Item-item based CF identifies items similar to those a user has already liked, and is according to some studies faster than the user-user based when there are more users than items. User-item based CF learns compact latent vector representations that capture the underlying preference structures reflected in the user-item preference matrix, through various machine learning techniques [2].

Another recommender system type is content-based filtering, which (like item-item based CF) recommends items similar to those a user has previously interacted with, based on metadata or textual features (like description, developer, genre, etc.). Hybrid systems combine collaborative and content-based methods to overcome limitations such as cold-start problems and data sparsity [4].

More recent types, such as context-aware and knowledge-graph-based systems, incorporate temporal, spatial, or semantic information to improve relevance, while deep learning models, such as Neural Collaborative Filtering (NCF) and reinforcement-learning recommenders, leverage neural architectures for nonlinear interaction modeling and dynamic adaptation [2]. These developments represent a shift from simple correlation-based prediction toward causality-aware and behaviorally adaptive recommendation frameworks.

Since the early 1990s, recommender systems have evolved into indispensable components of modern digital platforms, including e-commerce, streaming services, social media, and many other domains. E-commerce sites such as Amazon use recommender systems to increase product exposure, Netflix relies on them to suggest movies and shows, and Spotify organizes personalized playlists. Similar techniques are used in job-matching services, social network connections, and even healthcare recommendations. The growing importance of recommender systems stems from their many roles, ranging from improving user experience through personalization, to boosting long-term user engagement and/or sales generating economic value [4].

Precisely because recommender systems are used across such a wide range of domains and purposes, there is no single model that performs best in all situations. The underlying assumptions, data requirements, and computational strategies vary

considerably, making fair comparison across models and datasets challenging. That is exactly what motivates this study, which aims to apply a standardized evaluation framework to compare different collaborative filtering approaches across varying datasets.

The aim of this project is to answer the research question:

How do different collaborative filtering approaches compare in terms of recommendation accuracy, scalability, and robustness across different benchmark datasets?

- **Accuracy:** Which of the models gives the best recommendation accuracy on each dataset?
- **Scalability:** How do the models compare in training and prediction time across the datasets of different sizes?
- **Robustness:** How stable are the results of each model when tested on different types of datasets? Do the models perform differently on datasets with explicit ratings compared to implicit data?

The specific models compared in this thesis are ItemKNN, BPR, NeuMF, and GCMC, evaluated on the datasets amazon_all_beauty, ml-100k, douban, and jester, where each dataset is processed using three distinct preprocessing strategies.

2 Background

Recommender systems have evolved from experimental information-filtering tools into complex machine learning ecosystems powering today’s digital economy. Their development has been driven by advances in data availability, computational power, and algorithmic innovation.

2.1 History

Recommender systems have its origins in the early 1990s, when researchers began exploring information filtering to manage the growing flow of digital content. One of the earliest examples was Tapestry (1992)[4], an email filtering system that introduced the principle of collaborative filtering relying on collective human feedback to recommend relevant items. This concept was soon automated in GroupLens (1994)[4], a news recommendation system that used explicit user ratings, to predict preferences of unseen items [2][4].

By the late 1990s, recommender systems had expanded from research prototypes to commercial implementations, with Amazon.com being one of the first large-scale adopters of recommendation technology [4]. These early recommendation systems relied on methods that operated directly on the raw user-item interaction data at prediction time, rather than learning explicit model parameters. Approaches such as K-Nearest-Neighbor compared users or items by computing similarity metrics [2].

2.2 Modern methods

As datasets grew larger, scalability and sparsity issues were addressed by reducing the dimensionality of the user–item interaction data. In the early 2000s, latent factor models such as matrix factorization began to dominate the field. By representing users and items as dense embeddings in a shared latent space, these models were able to capture hidden preference patterns. Research on these methods intensified significantly during the Netflix Prize competition (2006–2009) [4] [2].

In the 2010s, recommender systems increasingly adopted fully machine-learning driven approaches, building on probabilistic and optimization-based frameworks such as Factorization Machines and Bayesian Personalized Ranking [2] [9].

Simultaneously, researchers began integrating multiple information sources, such as metadata, multi-criteria ratings, social networks, demographic attributes, temporal information, and other contextual data, into hybrid recommender systems. This approach helped address limitations of earlier linear models, including the cold-start problem [4] [2].

Gradually, recommender systems research became increasingly reliant on standardized offline evaluation metrics such as Precision and Recall to quantify predictive accuracy. These computational measures enabled comparative analysis across models and datasets, but they also introduced methodological and reproducibility challenges, as differences in dataset splits, preprocessing choices, and training pipelines often produced inconsistent results across studies [4].

The rapid advancement of deep learning techniques fundamentally reshaped recommender systems research after 2016. Neural Collaborative Filtering frameworks

replaced the traditional inner product used in matrix factorization with nonlinear neural architectures capable of learning more complex user–item interactions [2]. Models such as NeuMF further introduced multi-layer perceptrons to capture non-linear relationships [3], while additional neural mechanisms were developed to model the evolution of user interests and the relevance between a user’s historical interactions and a target item [2].

At the same time, graph-based recommendation models emerged as a powerful extension of collaborative filtering. By representing users and items as nodes within a graph structure, Graph Neural Network approaches such as Graph Convolutional Matrix Completion (GCMC) enabled more expressive learning of relational dependencies [2] [10].

2.3 Evaluation and present challenges

However, as model complexity increased, concerns arose regarding reproducibility and comparability. To address these issues, open-source frameworks such as RecBole were developed to standardize data processing, model training, and evaluation. Introduced as a unified, comprehensive, and efficient framework for recommendation algorithms, RecBole integrates more than 70 models and 28 benchmark datasets within a PyTorch-based pipeline [13]. The subsequent RecBole 2.0 release and the Recent Advances in RecBole extensions further expanded support for cutting-edge architectures such as Transformers and GNNs, while emphasizing reproducibility, flexibility, and efficiency [12] [11].

Despite the progress, several challenges remain. Scalability remains a critical factor, as modern applications must serve billions of users and items in real time. Increasing demands for explainability, fairness, and diversity further require algorithms to not only perform well, but also to justify their outputs transparently and objectively [4].

3 Methods

This section presents the mathematical formulations and training objectives of the four collaborative filtering models compared in this thesis: ItemKNN, BPR, NeuMF, and GCMC.

Throughout the section, the following definitions will be used to ensure consistency and avoid repeated definitions.

Let:

- $\mathcal{U}$ and $\mathcal{I}$ denote the sets of all users and all items, respectively.
- $u \in \mathcal{U}$ and $i \in \mathcal{I}$ denote a user and item, respectively.
- $\mathcal{U}_i \subseteq \mathcal{U}$ denote the set of users who have interacted with item i .
- $\mathcal{I}_u \subseteq \mathcal{I}$ denote the set of items with which user u has interacted.
- $R \in \mathbb{R}^{n \times m}$ denote the user-item rating / interaction matrix, where $n = |\mathcal{U}|$ and $m = |\mathcal{I}|$.
- $P \in \mathbb{R}^{n \times E}$ and $Q \in \mathbb{R}^{m \times E}$ denote the trainable user and item embedding matrices, respectively, where E is the embedding dimension.
- P_u and Q_i denote the embedding vectors for user u and item i .

3.1 ItemKNN

Item-based top-N recommendation is a collaborative filtering algorithm that estimates a user's preference for an item based on item-item similarity computed from the user-item interaction matrix R [1].

For each pair of items (i, j) , ItemKNN computes a similarity score based on the users who have interacted with both items.

$$\text{sim}(i, j) = \cos(R_{*,i}, R_{*,j}) = \frac{R_{*,i} \bullet R_{*,j}}{\|R_{*,i}\|_2 \cdot \|R_{*,j}\|_2}, \quad (1)$$

where $R_{*,i} = [R_{1,i}, R_{2,i}, \dots, R_{n,i}]$ and $R_{*,j} = [R_{1,j}, R_{2,j}, \dots, R_{n,j}]$ denote the item column vectors across all users, and $\bullet$ denotes the vector dot-product operation.

In practice, a regularized variant of equation 1 may be applied:

$$\text{sim}(i, j)_{\text{Recbole}} = \frac{R_{*,i} \bullet R_{*,j}}{\|R_{*,i}\|_2 \cdot \|R_{*,j}\|_2 + \text{shrink} + \epsilon}, \quad (2)$$

where *shrink* is a regularization hyperparameter that penalizes similarity estimates when two items share few co-interacting users, and ϵ is a small soothing factor like $1e^{-6}$ to prevent division by zero [7].

For each item i , only the k most similar items are retained, yielding a sparse item-item similarity matrix $S \in [0, 1]^{m \times m}$. From the use of cosine similarity, S is symmetric:

$$S_{i,j} = S_{j,i}, \quad \forall i, j \in \{1, \dots, m\}.$$

However, other potentially non-symmetric similarity measures may be used as well [1].

For each item i that a user has not interacted with, its recommendation strength is computed as the sum of similarities between i and all items previously interacted with by the user.

$$\hat{y}_{u,i} = \sum_{j \in \mathcal{I}_u} S_{i,j}.$$

For each user u , the model ranks all unseen items by their computed scores $\hat{y}_{u,i}$, and the top- N items with the highest scores are recommended.

3.2 BPR

Bayesian Personalized Ranking is a pairwise learning approach (designed for implicit feedback recommendation tasks). It optimizes a personalized ranking of items for each user by modeling relative preferences between item pairs [9].

The method constructs a training set of pairwise preference triples:

$$D = \{ (u, i, j) \mid u \in \mathcal{U}, i \in \mathcal{I}_u, j \in \mathcal{I} \setminus \mathcal{I}_u \}. \quad (3)$$

Each triple (u, i, j) encodes the assumption that user u prefers item i over item j , written as $i >_u j$, since u has interacted with i but not with j .

The model parameters consist of the user and item embedding matrices P and Q .

The predicted preference score is defined by:

$$\hat{y}_{u,i} = \sum_{f=1}^E P_{u,f} \cdot Q_{i,f} = P_u \bullet Q_i, \quad (4)$$

where $\bullet$ denotes the vector dot product operation.

For a sampled triple (u, i, j) , the predicted preference difference is defined by:

$$\hat{y}_{u,i,j} = \hat{y}_{u,i} - \hat{y}_{u,j}. \quad (5)$$

The loss function minimizes the negative log-probability of the pairwise preferences:

$$\mathcal{L} = - \sum_{(u,i,j) \in D} \log \sigma(\hat{y}_{u,i,j}), \quad (6)$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the logistic sigmoid [5].

Training is performed using stochastic gradient descent. Triples (u, i, j) are sampled uniformly at random from D , and the corresponding parameter components $\Phi \in \{P_u, Q_i, Q_j\}$ are updated using their derivatives:

$$\frac{\partial \mathcal{L}}{\partial \Phi} = \frac{\partial \mathcal{L}}{\partial \hat{y}_{u,i,j}} \cdot \frac{\partial \hat{y}_{u,i,j}}{\partial \Phi} = (\sigma(\hat{y}_{u,i,j}) - 1) \cdot \begin{cases} Q_i - Q_j & \text{if } \Phi = P_u, \\ P_u & \text{if } \Phi = Q_i, \\ -P_u & \text{if } \Phi = Q_j, \\ 0 & \text{else} \end{cases}$$

[9].

3.3 NeuMF

Neural Matrix Factorization expands traditional matrix factorization to a learnable non-linear function. The model jointly learns user and item representations through two parallel components: a generalized matrix factorization (GMF) branch capturing linear interactions, and a multi-layer perceptron (MLP) branch capturing non-linear feature interactions [3].

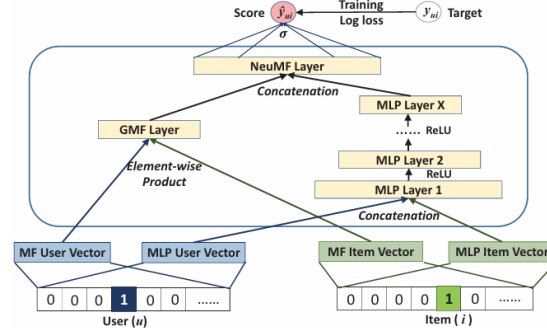


Figure 3: Neural matrix factorization model

Figure 1: A visual representation of the model from [3]

NeuMF operates on implicit-feedback data, where a user-item interaction is labeled as $y_{u,i} = [0, 1]$.

The two branches are parameterized by separate embedding matrices [8]:

$$\begin{aligned} P^{GMF} &\in \mathbb{R}^{n \times E^{GMF}}, & Q^{GMF} &\in \mathbb{R}^{m \times E^{GMF}}, \\ P^{MLP} &\in \mathbb{R}^{n \times E^{MLP}}, & Q^{MLP} &\in \mathbb{R}^{m \times E^{MLP}}, \end{aligned}$$

for GMF and MLP respectively, where E^{GMF} and E^{MLP} denote the potentially different embedding dimensions.

GMF branch:

For each user-item pair, the branch computes the interaction vector using element-wise multiplication:

$$\phi_{u,i}^{GMF} = P_u^{GMF} \odot Q_i^{GMF}, \quad (7)$$

where $\odot$ denotes the element-wise (Hadamard) product.

MLP branch:

This branch models higher-order, non-linear interactions. It consists of a stack of hidden layers z_l for $l = 1, \dots, L$, where L is the number of layers. First, the user and item embeddings are concatenated

$$z_1 = \begin{bmatrix} P_u^{MLP} \\ Q_i^{MLP} \end{bmatrix}. \quad (8)$$

This vector is then passed through the remaining $L - 1$ layers:

$$z_l = a_l (W_l^T z_{l-1} + b_l) \quad , \quad l = 2, \dots, L, \quad (9)$$

where W_l and b_l denote the learnable weight matrix and bias vector of layer l , respectively, and $a_l(\cdot)$ denotes its activation function (e.g., ReLU).

The final hidden layer output is:

$$\phi_{u,i}^{MLP} = z_L. \quad (10)$$

Combination and prediction:

The model branches are combined by concatenating their last hidden layers (see 1):

$$\phi_{u,i}^{NeuMF} = \begin{bmatrix} \phi_{u,i}^{GMF} \\ \phi_{u,i}^{MLP} \end{bmatrix}. \quad (11)$$

It is then passed through a final linear layer, followed by a sigmoid activation to obtain a probability-like score:

$$\hat{y}_{ui} = \sigma(w^T \phi_{u,i}^{NeuMF} + b), \quad (12)$$

where w and b denote the learnable weight matrix and bias vector of the final linear layer and $\sigma(x) = \frac{1}{1+e^{-x}}$ is the logistic sigmoid function ensuring $\hat{y}_{ui} \in (0, 1)$ [8].

Training objective:

NeuMF is trained using binary cross-entropy loss:

$$\mathcal{L} = - \sum_{(u,i)} (y_{u,i} \log \hat{y}_{u,i} + (1 - y_{u,i}) \log(1 - \hat{y}_{u,i})), \quad (13)$$

where $y_{u,i}$ and $\hat{y}_{u,i}$ are the true and predicted labels respectively. For any model parameter θ (such as an embedding vector in P^{GMF} , Q^{GMF} , P^{MLP} , Q^{MLP} , or a weight or bias in the MLP or final layer), the gradient $\partial \mathcal{L} / \partial \theta$ is computed by back-propagation. Parameters are updated with stochastic gradient descent or Adam:

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta},$$

where η is the learning rate [8].

3.4 GCMC

Graph Convolutional Matrix Completion models the recommendation problem as a link prediction task on a bipartite graph consisting of user and item nodes, where edges represent observed ratings or interactions [10].

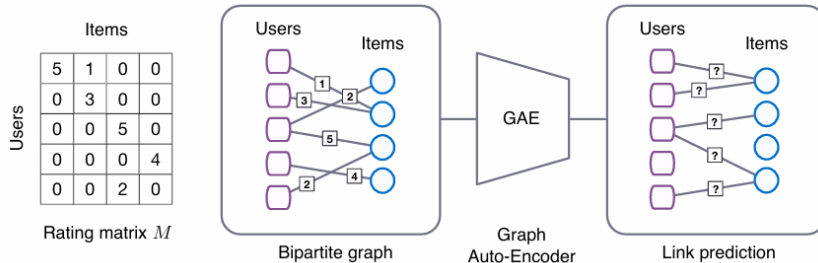


Figure 1: *Left*: Rating matrix M with entries that correspond to user-item interactions (ratings between 1-5) or missing observations (0). *Right*: User-item interaction graph with bipartite structure. Edges correspond to interaction events, numbers on edges denote the rating a user has given to a particular item. The matrix completion task (i.e. predictions for unobserved interactions) can be cast as a link prediction problem and modeled using an end-to-end trainable graph auto-encoder.

Figure 2: A visual representation of the model from [10]

The corresponding graph consists of a set of nodes $\mathcal{U} \cup \mathcal{I}$, edges $\mathcal{E}$, and the set of observed ordinal rating values $\mathcal{R}$:

$$G = (\mathcal{U} \cup \mathcal{I}, \mathcal{E}, \mathcal{R}),$$

where

$$\mathcal{R} = \{r \mid r = R_{u,i} \text{ for some observed } (u, i)\}$$

is the set of unique ratings in R , and edges $(u, r, i) \in \mathcal{E}$ indicates that user u rated item i with rating $r \in \mathcal{R}$ (see figure 2).

The task of predicting the value of unobserved entries in R , i.e., missing edges in G , is accomplished through a graph auto-encoder model

$$f(X, A_{\mathcal{R}_{min}}, \dots, A_{\mathcal{R}_{max}}) = [P, Q],$$

followed by a pairwise decoder model

$$g(P, Q) = \hat{R},$$

where:

- For each rating $r \in \mathcal{R}$, an associated adjacency matrix $A_r \in \{0, 1\}^{n \times m}$ is defined by

$$A_{r_{u,i}} = \begin{cases} 1 & , \text{ if } (R_{u,i})_{level} = r \\ 0 & , \text{ otherwise} \end{cases}$$

- $X \in \mathbb{R}^{(n+m) \times D}$ is the input feature matrix of all nodes in the graph, where D denotes the dimensionality of the input features.
- $\hat{R}$ is the predicted rating matrix.

Encoder:

The encoder generates embeddings by passing messages $\mu_{j \rightarrow l, r}$ over the bipartite graph. For each rating $r \in \mathcal{R}$, messages are propagated from node j to the neighboring nodes l . Note, indices j and l may refer to either users or items; that is, j is an item if l is a user, and vice versa. A message is computed as:

$$\mu_{j \rightarrow l, r} = \frac{1}{c_{l,j}} W_r X_j, \quad (14)$$

where W_r is a trainable weight matrix specific to rating r , X_j is the feature vector of node j , and

$$c_{l,j} = \sqrt{|\mathcal{N}_l| |\mathcal{N}_j|}$$

is a symmetric normalization constant [6] with $\mathcal{N}_j$ denoting the set of neighbors to node j . Other non symmetric normalization constants can be used as well [10].

For a fixed node l , the messages from all neighboring nodes with rating r are aggregated by summation, and the rating-specific message vectors are accumulated using either vector summation or concatenation:

$$h_l = \sigma \left(\text{accum} \left(\sum_{j \in \mathcal{N}_{l,r}} \mu_{j \rightarrow l, r} \mid r \in \mathcal{R} \right) \right), \quad (15)$$

where $\sigma(\cdot) = \text{ReLU}(\cdot) = \max(0, \cdot)$ and $\mathcal{N}_{l,r}$ denote the neighbors of l connected via rating r .

A shared linear transformation produces the final embedding:

$$z_l = \sigma(Mh_l) \quad , \quad z_l = \begin{cases} P_u & , \text{ if } l = u \in \mathcal{U} \\ Q_i & , \text{ if } l = i \in \mathcal{I} \end{cases} \quad (16)$$

where $M \in \mathbb{R}^{|\mathcal{H}| \times E}$ is a shared trainable projection matrix mapping h_l to z_l .

Decoder:

For each user-item pair (u, i) , the decoder reconstructs the rating by applying a softmax function to obtain a probability distribution over rating values $r \in \mathbb{R}$:

$$\mathbb{P}(\hat{R}_{u,i} = r) = \frac{e^{P_u^\top S_r Q_i}}{\sum_{s \in \mathcal{R}} e^{P_u^\top S_s Q_i}} \quad (17)$$

where $S_r \in \mathbb{R}^{E \times E}$ is a trainable rating specific parameter matrix. The predicted rating is the expectation over this distribution:

$$\hat{R}_{u,i} = g(P_u, Q_i) = \mathbb{E}(r) = \sum_{r \in \mathcal{R}} r \mathbb{P}(\hat{R}_{u,i} = r) \quad (18)$$

Training objective:

The model is trained by minimizing the negative log-likelihood of the observed ratings:

$$\mathcal{L} = - \sum_{(u,i) | i \in \mathcal{I}_u} \sum_{r \in \mathcal{R}} \mathbb{I}(R_{u,i} = r) \log \mathbb{P}(\hat{R}_{u,i} = r), \quad (19)$$

where the indicator function:

$$\mathbb{I}(R_{u,i} = r) = \begin{cases} 1 & , \text{ if } R_{u,i} = r \\ 0 & , \text{ otherwise} \end{cases}$$

4 Experiments

In this section, the experimental setup is described, including the models, datasets, preprocessing strategies, evaluation protocol, and metrics used to compare model performance.

The experiments are conducted using the Python library *RecBole*, employing its implementations of the selected models and preprocessed datasets.

The specific models compared are Item-based top-N nearest neighbor (ItemKNN)¹, Bayesian Personalized Ranking (BPR)², Neural Matrix Factorization (NeuMF)³ and Graph Convolutional Matrix Completion (GCMC)⁴, evaluated on the datasets amazon.all.beauty⁵, ml-100k⁶, douban⁷ and jester⁸.

Each dataset is processed in three distinct ways:

expR: True explicit - processed as an explicit dataset with ratings.

impR: Explicit/implicit mix - processed as an implicit dataset while retaining the explicit rating values.

impB: True implicit - processed as an implicit dataset with binary labels.

Resulting in a total of 48 experiment configurations.

For ItemKNN, the source code [7] was altered slightly. In RecBole’s default implementation, the actual rating values in the user-item interaction matrix are ignored, and each observed interaction is treated as a binary event, i.e., $R_{u,i} = 1$ for all observed user-item interactions and 0 otherwise. For the experiments in this thesis, the source code was modified to retain the original interaction ratings. Specifically, line 183 in the original implementation was replaced.

Original implementation:

```
self.interaction_matrix =  
    dataset.inter_matrix(form="csr").astype(np.float32)
```

Modified implementation:

```
self.interaction_matrix = dataset.inter_matrix(form="csr",  
    value_field=config["RATING_FIELD"]).astype(np.float32)
```

Details of the experimental environment are provided in appendix A.

¹https://recbole.io/docs/user_guide/model/general/itemknn.html

²https://recbole.io/docs/user_guide/model/general/bpr.html

³https://recbole.io/docs/user_guide/model/general/neumf.html

⁴https://recbole.io/docs/user_guide/model/general/gcmc.html

⁵<https://drive.google.com/drive/folders/1b3F9F0i8X8BqUUZOE2Aii4Ud8kEZvQbP>

⁶https://drive.google.com/drive/folders/10kDVEqetv0rtbuWebx14y1J1Z_YjjfWj

⁷<https://drive.google.com/drive/folders/17Kygc6s0sbveLSWMMVEKGmxt3iL6NQs>

⁸<https://drive.google.com/drive/folders/1vrJnXIwol2M9w3aAbJl4j-RcyrMlIPYz>

4.1 Data

This section briefly introduces the datasets used in the experiments. The dimensionality of the raw datasets is presented in Table 1, where

$$Sparsity = \frac{Zero\ elements}{Total\ elements} = \frac{(\#User \cdot \#Item) - \#Interaction}{(\#User \cdot \#Item)}$$

measures the proportion of missing interactions in the user-item interaction matrix.

amazon_all_beauty is a dataset containing reviews from the Beauty category of the Amazon Reviews dataset, collected in 2023.

ml-100k is a dataset containing movie ratings collected by the GroupLens Research Project via the MovieLens website between September 1997 and April 1998.

douban is a dataset containing reviews from the Douban Movie website, a Chinese platform that allows users to rate and share opinions on 28 movies.

jester is a dataset containing continuous ratings of 100 jokes, collected between April 1999 and May 2003.

Dataset	#User	#Item	#Interaction	Sparsity	#Unique Ratings	Rating Range
amazon_all_beauty	631,986	112,565	693,929	99.9990%	5	[1, 5]
ml-100k	943	1,682	100,000	93.70%	5	[1, 5]
douban	738,701	28	2,125,056	89.73%	5	[1, 5]
jester	73,421	101	4,136,360	44.22%	412	[-10, 10]

Table 1: Raw explicit data

4.2 Data processing

This section describes the processing steps applied to the datasets.

The datasets are transformed into the three dataset types described earlier and split into training, validation, and test sets. A common evaluation protocol for recommender systems is leave-one-out splitting, either based on the most recent interaction [1, 3] or by random sampling [9]. Since not all of the datasets contain timestamps, uniform random sampling is used to ensure consistent and comparable splits across all datasets..

The datasets are filtered using the following procedure:

The datasets are loaded, keeping only the **user_id**, **item_id** and **rating** columns. Duplicate interactions (identical user-item pairs) are removed, keeping only the interaction with the highest rating.

For the datasets **impR** and **impB**, only positive interactions are retained. This is done by removing interactions with ratings below 75% of the rating scale. For example, for the Jester dataset, Remove if: $rating < (-10) + (10 - (-10)) \cdot 0.75 = 5$. Users with fewer than five interactions are removed. At least three interactions are required for leave-one-out splitting; a higher threshold could be used to better mitigate the cold-start problem, at the cost of reduced data density.

For users who have interacted with all items, one random interaction is removed to enable negative sampling during evaluation.

The datasets are split using leave-one-out with uniform random sampling, as described above.

For the dataset **impB** only, all remaining ratings are converted to 1, indicating implicit positive feedback.

Finally, the datasets are saved as training, validation, and test sets.

The full processing implementation is provided in Appendix 2.

The dimensionality of the processed explicit dataset expR is presented in Table 2, while the processed implicit datasets impR and impB are presented in Table 3. Table 3 reports the retained rating values for impR, whereas all labels in impB are equal to 1 (True implicit).

Dataset	#User	#Item	#Interaction	Sparsity	#Unique Ratings	Rating Range
amazon_all_beauty	1,471	6,801	13,944	99.86%	5	[1,5]
ml-100k	943	1,682	100,000	93.70%	5	[1,5]
douban	108,663	28	967,452	68.20%	5	[1,5]
jester	73,421	100	4,122,244	43.85%	412	[-10, 10]

Table 2: Processed explicit datasets

Dataset	#User	#Item	#Interaction	Sparsity	#Unique Ratings	Rating Range
amazon_all_beauty	998	4,784	9,712	99.80%	2	[4, 5]
ml-100k	938	1,447	55,361	95.92%	2	[4, 5]
douban	54,410	28	411,656	72.98%	2	[4, 5]
jester	50,097	100	1,013,397	79.77%	104	[5, 10]

Table 3: Processed implicit datasets

4.3 Setup

This section describes the experimental configuration and evaluation procedure.

Due to resource limitations, no hyperparameter tuning is performed; all models are used out of the box with their default settings. Examples of log files containing the applied parameters are provided in Appendix E.

Since the chosen experimental platform does not support the simultaneous use of rating-based metrics⁹ and ranking-based metrics¹⁰, a decision must be made when evaluating the models. As the experiments include both explicit and implicit datasets, rating-based evaluation is not applicable in all cases, since implicit data does not contain explicit rating values. Consequently, evaluation is performed using ranking-based metrics, which are compatible with both data types and reflect common real-world recommendation settings, enabling comparable results across all experiments.

The chosen evaluation Metrics@ K ¹¹, with cutoff $K \in \{10, 20\}$, are:

Precision: measuring how many of the recommended items are relevant.

Recall: measuring how many of the relevant items are successfully recommended.

MRR: (Mean Reciprocal Rank) measuring how early the first relevant item appears in the ranked list.

Hit: (Hit rate also abbreviated HR) measuring whether at least one relevant item appears in the recommendation list.

NDCG: (Normalized Discounted Cumulative Gain) measuring ranking quality by rewarding relevant items appearing earlier in the list.

In this context, a relevant item is defined as an item with which the user has interacted.

Early stopping is applied during training, terminating training after five consecutive epochs without improvement in the validation metric, with a maximum of 50 epochs. The validation metric used for early stopping is NDCG@10.

During training, evaluation is performed using the setting `uni100`, applying uniform negative sampling with 100 negative items sampled per positive interaction. Final testing is performed using full item ranking.

⁹Rating-based metrics measure how accurately a model predicts explicit rating values.

¹⁰Ranking-based metrics measure how well a model orders items by relevance, i.e., whether relevant items appear among the top- K recommendations.

¹¹<https://recbole.io/docs/recbole/recbole.evaluator.metrics.html>

5 Results

This section presents the results of the experiments. The results are obtained under identical experimental conditions and presented for all model, dataset, and datatype combinations considered in the thesis.

To improve readability, efficiency and effectiveness measurements are organized into separate tables, with effectiveness further divided into evaluation metrics at cutoff values $K=10$ and $K=20$. Efficiency measurements are reported in Table 6, while effectiveness measurements at $K=10$ and $K=20$ are reported in Table 4 and Table 5, respectively. The effectiveness tables include visual highlighting to indicate relative performance across models, datasets, and datatypes¹². The efficiency table reports multiple measurements, for which the column headings have been shortened for readability¹³.

Figures 3 -10 contain learning curves for all evaluated models under the corresponding dataset-datatype configurations. The figures show the evolution of the normalized training loss and the validation metric used for early stopping (NDCG@10) as functions of training epochs. Markers indicate the epoch at which the highest validation performance is achieved, corresponding to the early-stopping criterion applied during training. Since the learning curves obtained for the impR and impB datatypes are nearly indistinguishable across datasets and models, only the curves for impB are shown in this section to conserve space. The learning curves for impR are provided in Appendix B for comparison.

¹²Bold values indicate the best performing model per metric within each dataset-datatype group. Bold model names denote the overall best-performing model for each dataset, based on mean ranks computed across all metrics and datatypes. Likewise underlined datatypes denote the datatype that achieves the best mean rank for each model across all metrics and datasets. For each metric, candidates are ranked within fixed dataset-datatype or dataset-model groups; these ranks are then averaged across metrics and across datatypes (for model selection) or datasets (for datatype selection), and the candidate with the lowest average rank is highlighted.

¹³Detailed explanation of efficiency measurement column headings:

Epochs: Number of training epochs executed for the given dataset-datatype-model configuration.

Train: Average training time per epoch, measured in seconds.

Eval: Average evaluation time per epoch, measured in seconds.

Params: Number of trainable model parameters.

FLOPs: Estimated computational complexity, as reported by RecBole. This value represents a precomputed estimate based on model complexity and dataset size and does not correspond to the actual number of floating-point operations per second.

CPU: Average cpu utilization during training, measured in percent.

GPU: Average gpu utilization during training, measured in percent.

MEM: Average memory utilization during training, measured in percent.

Dataset	Datatype	Model	Recall@10	Precision@10	MRR@10	Hit@10	NDCG@10
amazon_all_beauty	<u>expR</u>	ItemKNN	0.1856	0.0186	0.0751	0.1856	0.1008
amazon_all_beauty	<u>expR</u>	BPR	0.1863	0.0186	0.0786	0.1863	0.1036
amazon_all_beauty	<u>expR</u>	NeuMF	0.2080	0.0208	0.0721	0.2080	0.1034
amazon_all_beauty	<u>expR</u>	GCMC	0.2733	0.0273	0.1029	0.2733	0.1423
amazon_all_beauty	impR	ItemKNN	0.1824	0.0182	0.0750	0.1824	0.0999
amazon_all_beauty	impR	BPR	0.0972	0.0097	0.0301	0.0972	0.0454
amazon_all_beauty	impR	NeuMF	0.2004	0.0200	0.0630	0.2004	0.0946
amazon_all_beauty	impR	GCMC	0.2024	0.0202	0.0651	0.2024	0.0968
amazon_all_beauty	impB	ItemKNN	0.1804	0.0180	0.0751	0.1804	0.0995
amazon_all_beauty	impB	BPR	0.0972	0.0097	0.0301	0.0972	0.0454
amazon_all_beauty	impB	NeuMF	0.2004	0.0200	0.0630	0.2004	0.0946
amazon_all_beauty	impB	GCMC	0.2024	0.0202	0.0651	0.2024	0.0968
ml-100k	<u>expR</u>	ItemKNN	0.7932	0.0793	0.4519	0.7932	0.5331
ml-100k	<u>expR</u>	BPR	0.7784	0.0778	0.4310	0.7784	0.5140
ml-100k	<u>expR</u>	NeuMF	0.8091	0.0809	0.4568	0.8091	0.5406
ml-100k	<u>expR</u>	GCMC	0.7911	0.0791	0.4319	0.7911	0.5173
ml-100k	impR	ItemKNN	0.7878	0.0788	0.4080	0.7878	0.4980
ml-100k	impR	BPR	0.7655	0.0765	0.3931	0.7655	0.4810
ml-100k	impR	NeuMF	0.7846	0.0785	0.4092	0.7846	0.4984
ml-100k	impR	GCMC	0.7644	0.0764	0.3843	0.7644	0.4739
ml-100k	impB	ItemKNN	0.7836	0.0784	0.4079	0.7836	0.4970
ml-100k	impB	BPR	0.7655	0.0765	0.3931	0.7655	0.4810
ml-100k	impB	NeuMF	0.7846	0.0785	0.4092	0.7846	0.4984
ml-100k	impB	GCMC	0.7665	0.0767	0.3846	0.7665	0.4746
douban	<u>expR</u>	ItemKNN	0.7835	0.0783	0.3377	0.7835	0.4420
douban	<u>expR</u>	BPR	0.7321	0.0732	0.3020	0.7321	0.4024
douban	<u>expR</u>	NeuMF	0.7659	0.0766	0.3184	0.7659	0.4229
douban	<u>expR</u>	GCMC	0.7973	0.0797	0.3429	0.7973	0.4494
douban	impR	ItemKNN	0.8495	0.0850	0.3886	0.8495	0.4967
douban	impR	BPR	0.8049	0.0805	0.3557	0.8049	0.4608
douban	impR	NeuMF	0.8363	0.0836	0.3729	0.8363	0.4809
douban	impR	GCMC	0.8382	0.0838	0.3748	0.8382	0.4835
douban	impB	ItemKNN	0.8499	0.0850	0.3887	0.8499	0.4968
douban	impB	BPR	0.8049	0.0805	0.3557	0.8049	0.4608
douban	impB	NeuMF	0.8363	0.0836	0.3729	0.8363	0.4809
douban	impB	GCMC	0.8381	0.0838	0.3747	0.8381	0.4833
jester	<u>expR</u>	ItemKNN	0.5620	0.0562	0.4136	0.5620	0.4500
jester	<u>expR</u>	BPR	0.9221	0.0922	0.7863	0.9221	0.8203
jester	<u>expR</u>	NeuMF	0.8442	0.0844	0.6060	0.8442	0.6641
jester	<u>expR</u>	GCMC	0.9243	0.0924	0.8123	0.9243	0.8403
jester	impR	ItemKNN	0.5820	0.0582	0.2546	0.5820	0.3315
jester	impR	BPR	0.6204	0.0620	0.2576	0.6204	0.3425
jester	impR	NeuMF	0.6234	0.0623	0.2592	0.6234	0.3445
jester	impR	GCMC	0.6429	0.0643	0.2713	0.6429	0.3584
jester	impB	ItemKNN	0.5813	0.0581	0.2559	0.5813	0.3323
jester	impB	BPR	0.6204	0.0620	0.2576	0.6204	0.3425
jester	impB	NeuMF	0.6234	0.0623	0.2592	0.6234	0.3445
jester	impB	GCMC	0.6431	0.0643	0.2718	0.6431	0.3588

Table 4: Effectiveness results at $K = 10$. The highlighting scheme is described in footnote 12 page 15.

Dataset	Datatype	Model	Recall@20	Precision@20	MRR@20	Hit@20	NDCG@20
amazon_all_beauty	expR	ItemKNN	0.2882	0.0144	0.0817	0.2882	0.1261
amazon_all_beauty	<u>expR</u>	BPR	0.3005	0.0150	0.0863	0.3005	0.1321
amazon_all_beauty	<u>expR</u>	NeuMF	0.3433	0.0172	0.0814	0.3433	0.1376
amazon_all_beauty	<u>expR</u>	GCMC	0.3753	0.0188	0.1099	0.3753	0.1680
amazon_all_beauty	impR	ItemKNN	0.3006	0.0150	0.0829	0.3006	0.1294
amazon_all_beauty	impR	BPR	0.2084	0.0104	0.0376	0.2084	0.0733
amazon_all_beauty	impR	NeuMF	0.3176	0.0159	0.0710	0.3176	0.1240
amazon_all_beauty	impR	GCMC	0.3166	0.0158	0.0727	0.3166	0.1252
amazon_all_beauty	<u>impB</u>	ItemKNN	0.3046	0.0152	0.0834	0.3046	0.1304
amazon_all_beauty	impB	BPR	0.2084	0.0104	0.0376	0.2084	0.0733
amazon_all_beauty	impB	NeuMF	0.3176	0.0159	0.0710	0.3176	0.1240
amazon_all_beauty	impB	GCMC	0.3166	0.0158	0.0727	0.3166	0.1252
ml-100k	expR	ItemKNN	0.8971	0.0449	0.4592	0.8971	0.5595
ml-100k	<u>expR</u>	BPR	0.9046	0.0452	0.4401	0.9046	0.5463
ml-100k	<u>expR</u>	NeuMF	0.9205	0.0460	0.4647	0.9205	0.5690
ml-100k	<u>expR</u>	GCMC	0.9173	0.0459	0.4408	0.9173	0.5493
ml-100k	impR	ItemKNN	0.8977	0.0449	0.4155	0.8977	0.5257
ml-100k	impR	BPR	0.9041	0.0452	0.4032	0.9041	0.5166
ml-100k	impR	NeuMF	0.9062	0.0453	0.4180	0.9062	0.5296
ml-100k	impR	GCMC	0.8998	0.0450	0.3940	0.8998	0.5085
ml-100k	<u>impB</u>	ItemKNN	0.9019	0.0451	0.4161	0.9019	0.5270
ml-100k	impB	BPR	0.9041	0.0452	0.4032	0.9041	0.5166
ml-100k	impB	NeuMF	0.9062	0.0453	0.4180	0.9062	0.5296
ml-100k	impB	GCMC	0.8987	0.0449	0.3941	0.8987	0.5084
douban	expR	ItemKNN	0.9822	0.0491	0.3518	0.9822	0.4927
douban	<u>expR</u>	BPR	0.9704	0.0485	0.3188	0.9704	0.4630
douban	<u>expR</u>	NeuMF	0.9812	0.0491	0.3338	0.9812	0.4780
douban	<u>expR</u>	GCMC	0.9847	0.0492	0.3564	0.9847	0.4973
douban	impR	ItemKNN	0.9944	0.0497	0.3994	0.9944	0.5342
douban	impR	BPR	0.9817	0.0491	0.3685	0.9817	0.5062
douban	impR	NeuMF	0.9943	0.0497	0.3846	0.9943	0.5218
douban	impR	GCMC	0.9931	0.0497	0.3863	0.9931	0.5236
douban	<u>impB</u>	ItemKNN	0.9944	0.0497	0.3994	0.9944	0.5343
douban	impB	BPR	0.9817	0.0491	0.3685	0.9817	0.5062
douban	impB	NeuMF	0.9943	0.0497	0.3846	0.9943	0.5218
douban	impB	GCMC	0.9932	0.0497	0.3862	0.9932	0.5235
jester	expR	ItemKNN	0.6306	0.0315	0.4182	0.6306	0.4671
jester	<u>expR</u>	BPR	0.9423	0.0471	0.7877	0.9423	0.8254
jester	<u>expR</u>	NeuMF	0.9138	0.0457	0.6109	0.9138	0.6817
jester	<u>expR</u>	GCMC	0.9474	0.0474	0.8138	0.9474	0.8461
jester	impR	ItemKNN	0.7504	0.0375	0.2661	0.7504	0.3739
jester	impR	BPR	0.8023	0.0401	0.2704	0.8023	0.3886
jester	impR	NeuMF	0.8084	0.0404	0.2722	0.8084	0.3914
jester	impR	GCMC	0.8241	0.0412	0.2839	0.8241	0.4043
jester	<u>impB</u>	ItemKNN	0.7515	0.0376	0.2676	0.7515	0.3752
jester	impB	BPR	0.8023	0.0401	0.2704	0.8023	0.3886
jester	impB	NeuMF	0.8084	0.0404	0.2722	0.8084	0.3914
jester	impB	GCMC	0.8235	0.0412	0.2844	0.8235	0.4045

Table 5: Effectiveness results at $K = 20$. The highlighting scheme is described in footnote 12 page 15.

Dataset	Datatype	Model	Epochs	Train	Eval	Params	FLOPs	CPU	GPU	MEM
amazon_all_beauty	expR	ItemKNN	1	0.04	0.78	1	0	7.90	0.25	5.64
amazon_all_beauty	expR	BPR	50	0.04	0.47	529536	128	9.10	0.62	14.61
amazon_all_beauty	expR	NeuMF	14	0.10	0.51	1083969	24960	12.20	4.38	12.65
amazon_all_beauty	expR	GCMC	45	0.12	0.63	4177196	264768004	10.20	7.50	13.56
amazon_all_beauty	impR	ItemKNN	1	0.02	0.45	1	0	12.10	0.25	5.50
amazon_all_beauty	impR	BPR	9	0.03	0.32	370176	128	11.30	0.62	14.61
amazon_all_beauty	impR	NeuMF	14	0.08	0.36	765249	24960	10.30	4.38	12.65
amazon_all_beauty	impR	GCMC	11	0.08	0.41	2932196	185088004	9.50	7.50	13.50
amazon_all_beauty	impB	ItemKNN	1	0.03	0.67	1	0	10.00	0.25	5.44
amazon_all_beauty	impB	BPR	9	0.04	0.32	370176	128	9.20	0.62	14.61
amazon_all_beauty	impB	NeuMF	14	0.07	0.34	765249	24960	10.00	4.38	12.65
amazon_all_beauty	impB	GCMC	11	0.09	0.39	2932196	185088004	9.60	7.50	13.50
ml-100k	expR	ItemKNN	1	0.21	0.60	1	0	9.90	0.25	6.16
ml-100k	expR	BPR	37	0.26	0.40	168128	128	9.60	0.62	7.86
ml-100k	expR	NeuMF	35	0.45	0.40	361153	24960	11.00	4.50	7.80
ml-100k	expR	GCMC	43	0.61	0.45	1353696	84064004	7.90	7.50	8.98
ml-100k	impR	ItemKNN	1	0.14	0.60	1	0	8.30	0.25	6.09
ml-100k	impR	BPR	50	0.15	0.38	152768	128	11.80	0.62	14.68
ml-100k	impR	NeuMF	38	0.26	0.38	330433	24960	9.80	4.50	12.78
ml-100k	impR	GCMC	50	0.34	0.43	1233696	76384004	9.80	7.50	13.76
ml-100k	impB	ItemKNN	1	0.16	0.58	1	0	7.70	0.25	6.09
ml-100k	impB	BPR	50	0.14	0.36	152768	128	9.50	0.62	14.61
ml-100k	impB	NeuMF	38	0.24	0.35	330433	24960	9.40	4.50	12.78
ml-100k	impB	GCMC	50	0.33	0.41	1233696	76384004	9.30	7.50	13.63
douban	expR	ItemKNN	1	2.20	76.00	1	0	4.40	0.50	8.72
douban	expR	BPR	9	2.54	59.19	6956352	128	4.80	4.38	9.17
douban	expR	NeuMF	7	4.71	63.86	13937601	24960	5.00	7.50	8.91
douban	expR	GCMC	13	77.05	277.69	54386696	3478176004	4.10	53.25	8.85
douban	impR	ItemKNN	1	0.91	35.10	1	0	2.10	0.50	6.95
douban	impR	BPR	10	0.97	25.65	3484160	128	4.20	2.00	7.93
douban	impR	NeuMF	7	1.68	28.18	6993217	24960	5.20	4.50	7.99
douban	impR	GCMC	13	11.93	63.87	27260196	1742080004	6.20	22.75	8.85
douban	impB	ItemKNN	1	0.88	35.61	1	0	6.80	0.50	6.95
douban	impB	BPR	10	1.00	25.13	3484160	128	10.60	2.00	7.93
douban	impB	NeuMF	7	1.68	27.21	6993217	24960	6.40	4.50	7.99
douban	impB	GCMC	13	11.91	63.16	27260196	1742080004	4.70	22.75	8.85
jester	expR	ItemKNN	1	39.47	217.22	1	0	4.70	0.50	14.68
jester	expR	BPR	7	38.76	205.06	4705472	128	4.00	4.38	12.06
jester	expR	NeuMF	7	53.65	208.82	9435841	24960	3.30	7.50	12.65
jester	expR	GCMC	7	787.63	546.13	36801696	2352736004	5.10	56.50	12.78
jester	impR	ItemKNN	1	3.15	32.99	1	0	5.80	0.50	9.04
jester	impR	BPR	10	3.22	23.29	3212736	128	5.50	4.38	8.45
jester	impR	NeuMF	8	5.45	25.26	6450369	24960	5.90	7.50	8.26
jester	impR	GCMC	17	46.91	72.10	25139696	1606368004	6.10	53.25	8.52
jester	impB	ItemKNN	1	2.88	32.71	1	0	8.00	0.50	8.91
jester	impB	BPR	10	2.98	22.35	3212736	128	6.10	4.38	8.45
jester	impB	NeuMF	8	5.27	24.04	6450369	24960	6.60	7.50	8.32
jester	impB	GCMC	17	46.91	71.27	25139696	1606368004	5.70	53.25	8.52

Table 6: Efficiency results. Descriptions of the reported measurements are provided in footnote 13 page 15.

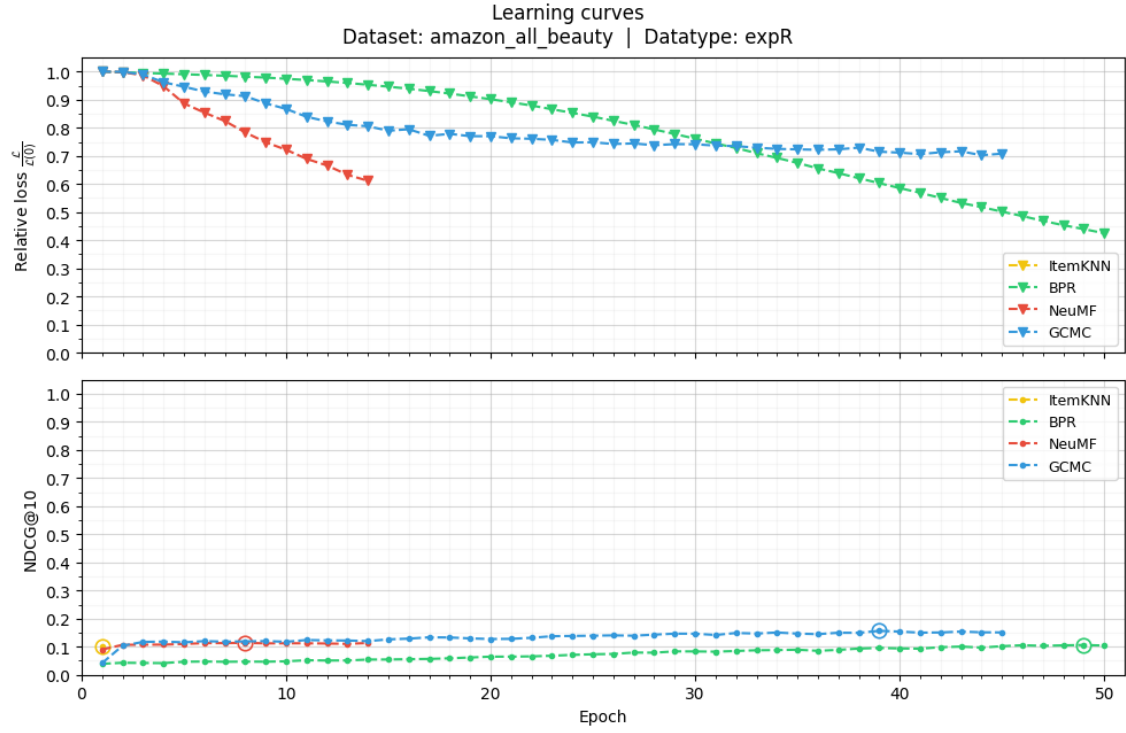


Figure 3: Learning curves for dataset `amazon_all_beauty` with datatype `expR`, showing relative training loss ($\blacktriangledown$) and the early-stopping validation metric NDCG@10 ($\bullet$) over epochs, with highest value ($\odot$) indication. Colors denote models.

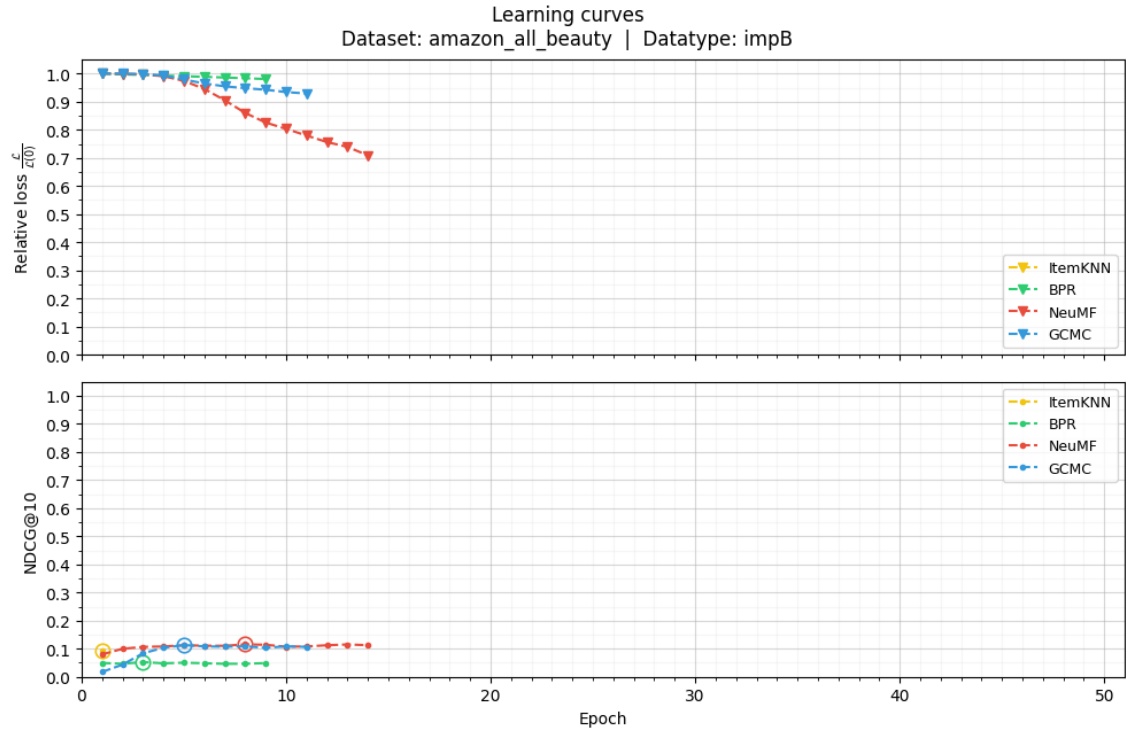


Figure 4: Learning curves for dataset `amazon_all_beauty` with datatype `impB`, showing relative training loss ($\blacktriangledown$) and the early-stopping validation metric NDCG@10 ($\bullet$) over epochs, with highest value ($\odot$) indication. Colors denote models.

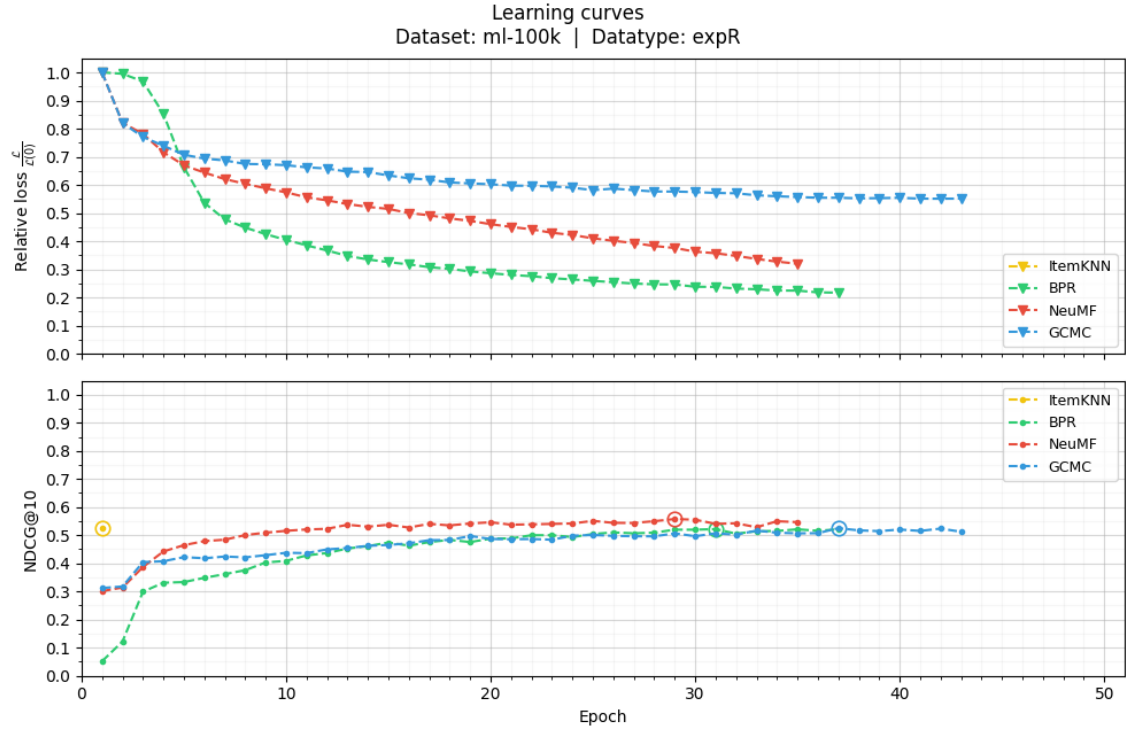


Figure 5: Learning curves for dataset `ml-100k` with datatype `expR`, showing relative training loss ($\blacktriangledown$) and the early-stopping validation metric NDCG@10 ($\bullet$) over epochs, with highest value ($\odot$) indication. Colors denote models.

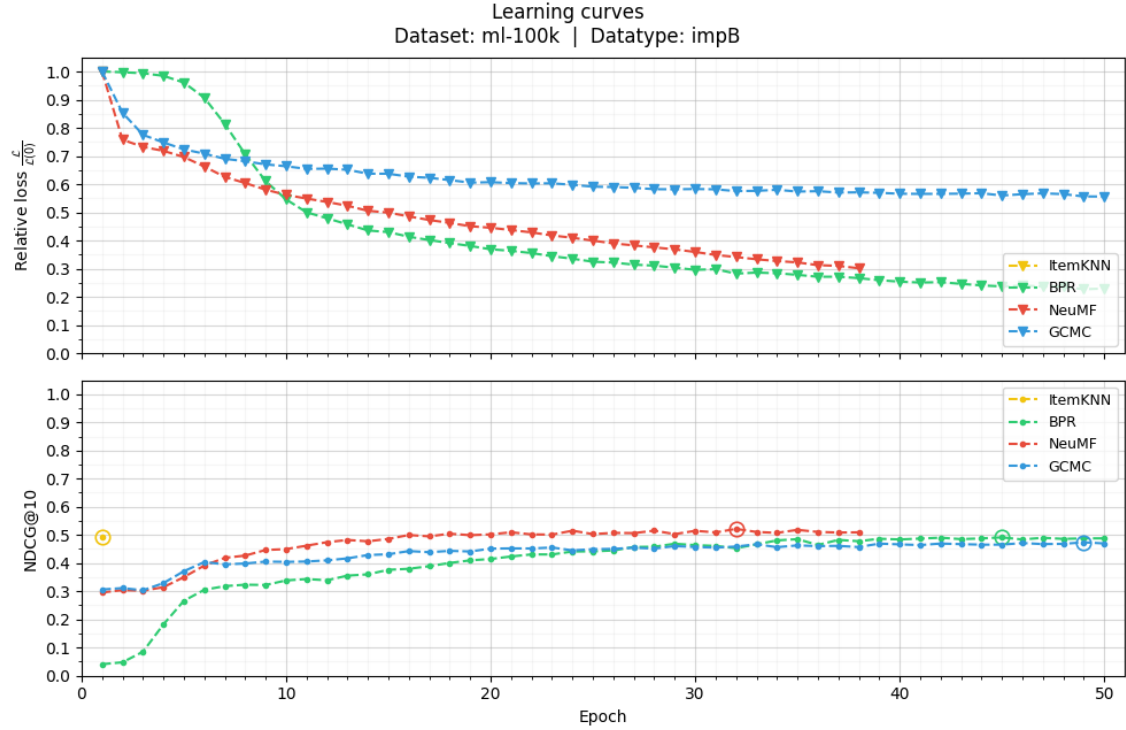


Figure 6: Learning curves for dataset `ml-100k` with datatype `impB`, showing relative training loss ($\blacktriangledown$) and the early-stopping validation metric NDCG@10 ($\bullet$) over epochs, with highest value ($\odot$) indication. Colors denote models.

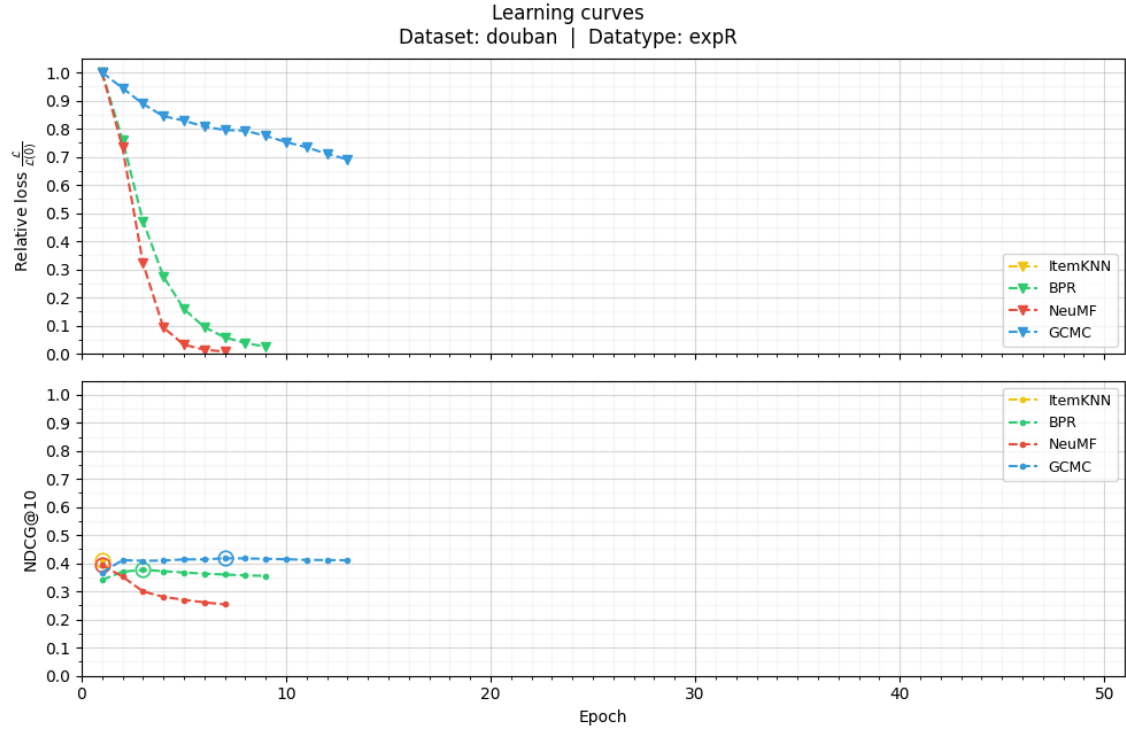


Figure 7: Learning curves for dataset **douban** with datatype **expR**, showing relative training loss ($\blacktriangledown$) and the early-stopping validation metric NDCG@10 ($\bullet$) over epochs, with highest value ($\odot$) indication. Colors denote models.

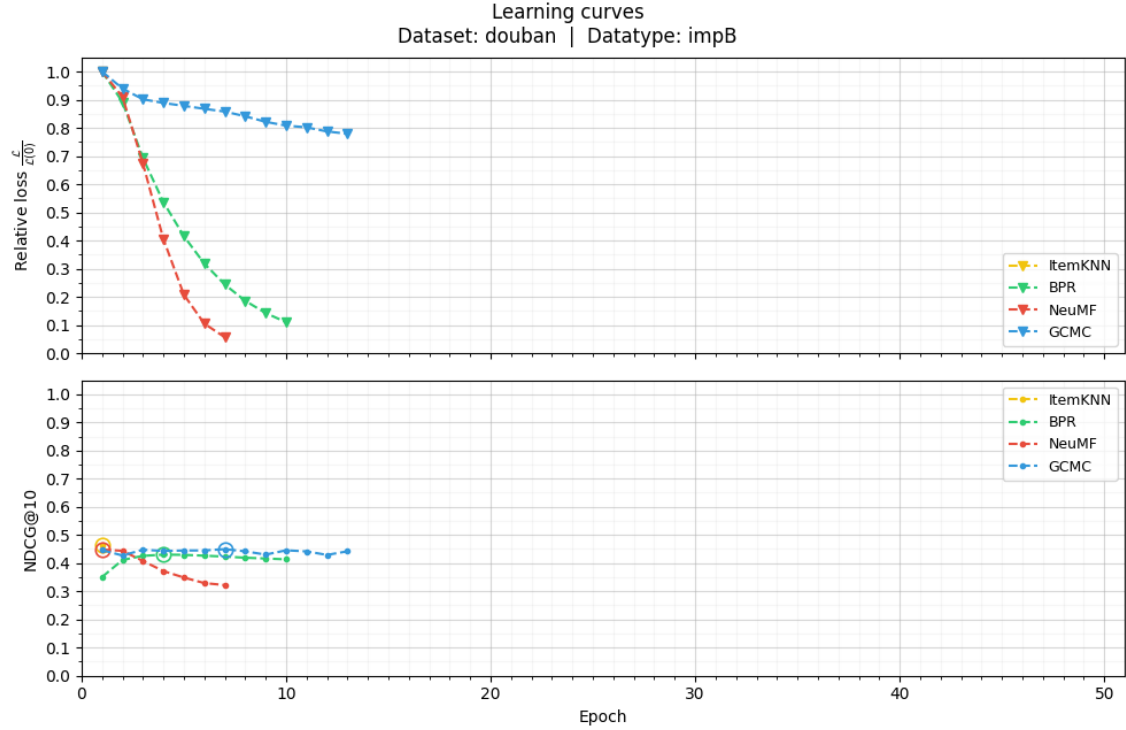


Figure 8: Learning curves for dataset **douban** with datatype **impB**, showing relative training loss ($\blacktriangledown$) and the early-stopping validation metric NDCG@10 ($\bullet$) over epochs, with highest value ($\odot$) indication. Colors denote models.

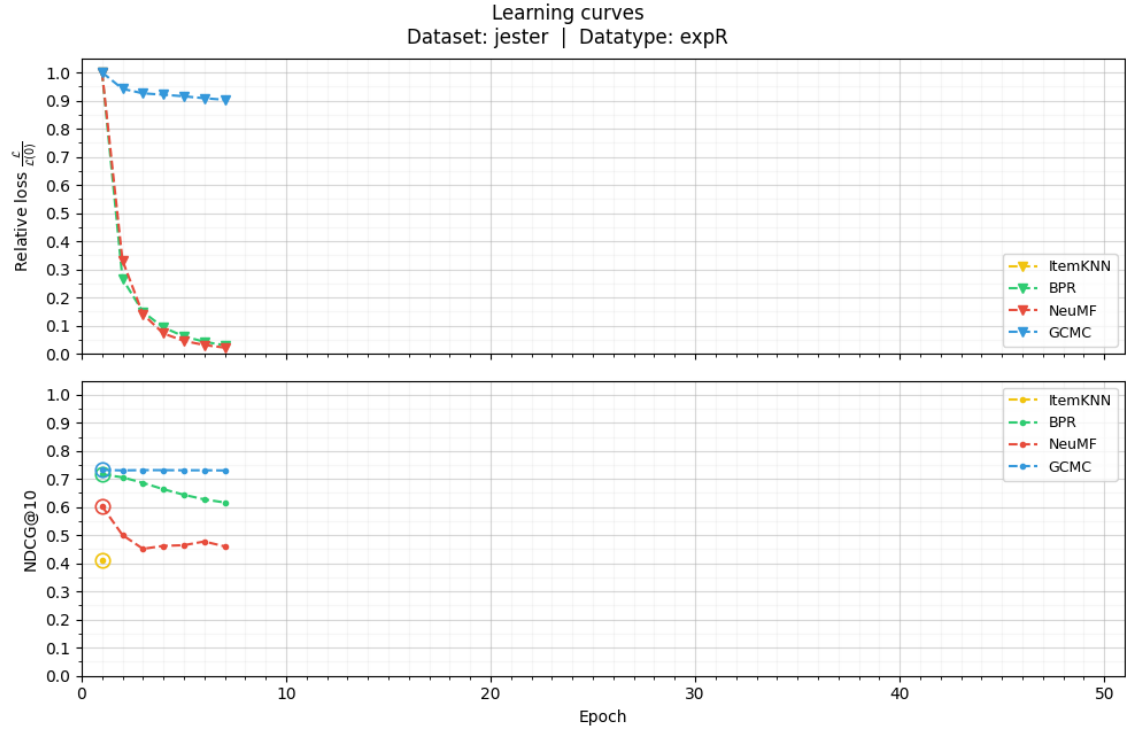


Figure 9: Learning curves for dataset `jester` with datatype `expR`, showing relative training loss ($\blacktriangledown$) and the early-stopping validation metric NDCG@10 ($\bullet$) over epochs, with highest value ($\odot$) indication. Colors denote models.

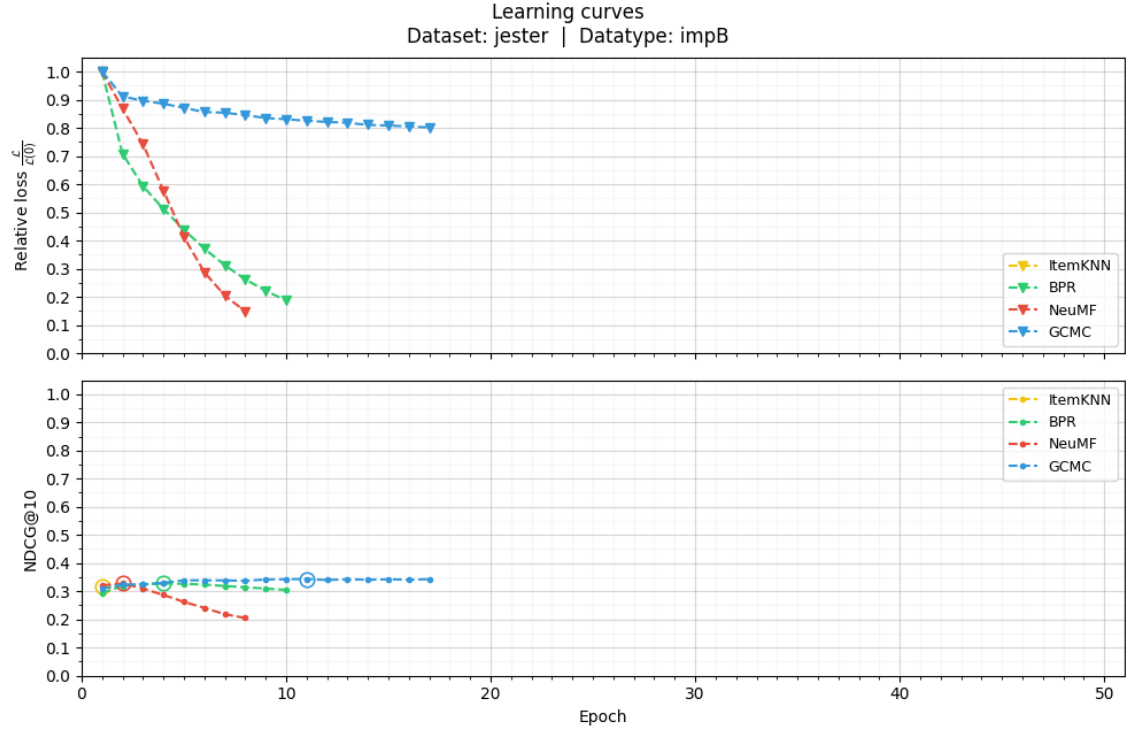


Figure 10: Learning curves for dataset `jester` with datatype `impB`, showing relative training loss ($\blacktriangledown$) and the early-stopping validation metric NDCG@10 ($\bullet$) over epochs, with highest value ($\odot$) indication. Colors denote models.

6 Discussion & Conclusion

This section discusses the experimental results presented in Section 5 and answers the research question:

How do different collaborative filtering approaches compare in terms of recommendation accuracy, scalability, and robustness across different benchmark datasets?

- **Accuracy:** Which models provide the strongest recommendation accuracy on each dataset?
- **Scalability:** How do the models compare in training and prediction time across datasets of different sizes?
- **Robustness:** How stable are the results across datasets and feedback types, and how do models behave on explicit versus implicit data?

Recommendation Accuracy.

The effectiveness results in Tables 4 and 5 show that no single model dominates across all dataset and datatype configurations. However, several consistent patterns emerge.

Across most datasets, GCMC achieves the strongest overall ranking performance, particularly on explicit datasets. On `amazon_all_beauty`, `douban`, and `jester` with the `expR` datatype, GCMC consistently attains the highest values at both $K = 10$ and $K = 20$. This shows that modeling user-item interactions as a graph allows GCMC to exploit relational structure that is not captured by simpler approaches.

NeuMF performs competitively on smaller and moderately sparse datasets, most notably on `ml-100k`, where it achieves the best or near best performance across metrics and datatypes. In contrast, NeuMF performs noticeably worse on `douban` and `jester` (particularly `jester expR`), where the number of items is small and user-item interactions are dense. This is also reflected in the learning curves shown in Figures 7–10, where the training loss continues to decrease while the validation metric NDCG@10 stagnates or declines. This indicates overfitting due to unnecessary model complexity, as simpler models already capture most of the signal.

ItemKNN performs strongly on `douban`, where the item set is very small and many users rate the same item pairs, resulting in dense co-occurrence. In this setting, ItemKNN is competitive with GCMC and achieves strong performance across datatypes. In contrast, ItemKNN performs very poorly on `jester`, despite dense interactions and a small item set. This may be caused by the more diverse rating distribution, yielding noisy similarity estimates and limiting the effectiveness of item-item similarity for recommendation.

BPR generally underperforms. However, it remains competitive on some implicit configurations and performs well on `jester expR`, particularly when computational cost is taken into account relative to NeuMF and GCMC.

Scalability and Efficiency.

The efficiency results in Table 6 show a clear trade-off between model complexity and computational cost.

ItemKNN is by far the most efficient model. It has no loss function or trainable parameters, and therefore incurs minimal training cost across all datasets. This makes ItemKNN well suited for large-scale or frequently updated systems. However, evaluation time is relatively high, as scoring each item requires aggregating similarities over all items a user has interacted with.

BPR and NeuMF exhibit moderate scalability. BPR consistently trains faster than NeuMF due to its simpler architecture, while NeuMF incurs higher computational cost from its neural components. For both models, training and evaluation cost grows roughly linearly with the number of observed interactions and the size of the item set, and does not increase disproportionately on larger datasets.

GCMC is the least scalable model. On douban and jester, GCMC requires substantially higher training time, FLOPs, and GPU utilization than the other models. While GCMC achieves the highest accuracy overall, the associated computational cost limits its applicability in large scale or resource constrained settings.

Robustness Across Datasets and Datatypes.

The results show high robustness across implicit datatypes. For all models and datasets, performance differences between impR and impB are negligible. This indicates that there is little to no benefit in retaining explicit rating values in implicit settings.

In general, the models perform very closely within each dataset. The only clear outlier is ItemKNN on jester, where it performs significantly worse than the other models. GCMC achieves the highest peak performance at the cost of scalability, while NeuMF represents a middle ground between model expressiveness and efficiency.

Conclusion.

The results show that the effectiveness, scalability, and robustness of the models depend strongly on the characteristics of the dataset.

GCMC provides the highest accuracy at a high computational cost, making it suitable when GPU resources and long training times are available.

NeuMF offers a balance between accuracy and efficiency.

BPR provides a lightweight alternative with a reasonable accuracy, cost tradeoff when training time is constrained.

ItemKNN serves as a strong baseline, delivering stable and competitive performance at a very low cost, and is well suited for settings where resources are limited or the model must be updated frequently.

Overall, the results confirm that there is no universally optimal collaborative filtering model. Model selection should instead be guided by the properties of the data, available computational resources, and the intended application setting.

7 References

- [1] Mukund Deshpande and George Karypis. Item-based top-n recommendation algorithms. *ACM Transactions on Information Systems*, 22(1):143–177, 2004. doi:[10.1145/963770.963776](https://doi.org/10.1145/963770.963776).
- [2] Zhenhua Dong, Zhe Wang, Jun Xu, Ruiming Tang, and Jirong Wen. A brief history of recommender systems. *arXiv:2209.01860v1*, 2022. doi:[10.48550/arXiv.2209.01860](https://doi.org/10.48550/arXiv.2209.01860).
- [3] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. *WWW '17: Proceedings of the 26th International Conference on World Wide Web*, pages 173–182, 2017. doi:[10.1145/3038912.3052569](https://doi.org/10.1145/3038912.3052569).
- [4] Dietmar Jannach, Pearl Pu, Francesco Ricci, and Markus Zanker. Recommender systems: Past, present, future. *AI Magazine*, 42(3):3–6, 2021. Downloaded from: <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/18139>. doi:[10.1609/aimag.v42i3.18139](https://doi.org/10.1609/aimag.v42i3.18139).
- [5] Recbole. Bpr source code. URL: https://recbole.io/docs/_modules/recbole/model/general_recommender/bpr.html.
- [6] Recbole. Gcmc source code. URL: https://recbole.io/docs/_modules/recbole/model/general_recommender/gcmc.html.
- [7] Recbole. Itemknn source code. URL: https://recbole.io/docs/_modules/recbole/model/general_recommender/itemknn.html.
- [8] Recbole. Neumf source code. URL: https://recbole.io/docs/_modules/recbole/model/general_recommender/neumf.html.
- [9] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. Bayesian personalized ranking from implicit feedback. *UAI '09: Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*, 22:452–461, 2009. URL: <https://dl.acm.org/doi/10.5555/1795114.1795167>.
- [10] Rianne van den Berg, Thomas N. Kipf, and Max Welling. Graph convolutional matrix completion. *arXiv:1706.02263*, 2017. doi:[10.48550/arXiv.1706.02263](https://doi.org/10.48550/arXiv.1706.02263).
- [11] Lanling Xu, Zhen Tian, Gaowei Zhang, Lei Wang, Junjie Zhang, Bowen Zheng, Yifan Li, Yupeng Hou, Xingyu Pan, Yushuo Chen, Wayne Xin Zhao, Xu Chen, and Ji-Rong Wen. Recent advances in recbole: Extensions with more practical considerations, 2022. doi:[10.48550/arXiv.2211.15148](https://doi.org/10.48550/arXiv.2211.15148).
- [12] Wayne Xin Zhao, Yupeng Hou, Xingyu Pan, Chen Yang, Zeyu Zhang, Zihan Lin, Jingsen Zhang, Shuqing Bian, Jiakai Tang, Wenqi Sun, et al. Recbole 2.0: Towards a more up-to-date recommendation library. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pages 4722–4726, 2022. doi:[10.1145/3511808.3557680](https://doi.org/10.1145/3511808.3557680).
- [13] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, Yingqian Min, Zhichao Feng, Xinyan Fan, Xu Chen, Pengfei Wang, Wendi Ji, Yaliang Li, Xiaoling

Wang, and Ji-Rong Wen. Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In *CIKM*, pages 4653–4664. ACM, 2021. [doi:10.1145/3459637.3482016](https://doi.org/10.1145/3459637.3482016).

A Experimental environment

This section describes the environment in which the experiments are run.

A.1 Hardware environment

This section describes the hardware configuration.

- **CPU:** AMD Ryzen 7 6800H
- **GPU:** NVIDIA GeForce RTX 3070 Ti Laptop
- **RAM:** 16 GB DDR5 (15,3 GB usable)
- **Storage:** 1 TB NVMe SSD

A.2 Software environment

This section describes the software configuration.

- **Operating System:** Windows 11 Home
 - Version 25H2
 - OS build 26200.7462
 - Feature Experience Pack 1000.26100.275.0
- **Python Version:** Python 3.11.13

The Python libraries used are specified in the requirements.txt below:

Listing 1: requirements.txt

```
1 #python=3.11.13
2 #pip install --upgrade torch==2.3.0 --index-url
   https://download.pytorch.org/whl/cu121
3 torch==2.3
4 scipy==1.10.1
5 recbole==1.2.1
6 matplotlib==3.10.6
7 ipywidgets==8.1.7
```

To enable GPU support, PyTorch is upgraded to the CUDA 12.1 GPU-enabled version with.

```
pip install --upgrade torch==2.3.0 --index-url
https://download.pytorch.org/whl/cu121
```

B impR learning curves

In this section, the learning curves for the impR datatype are provided for comparison.

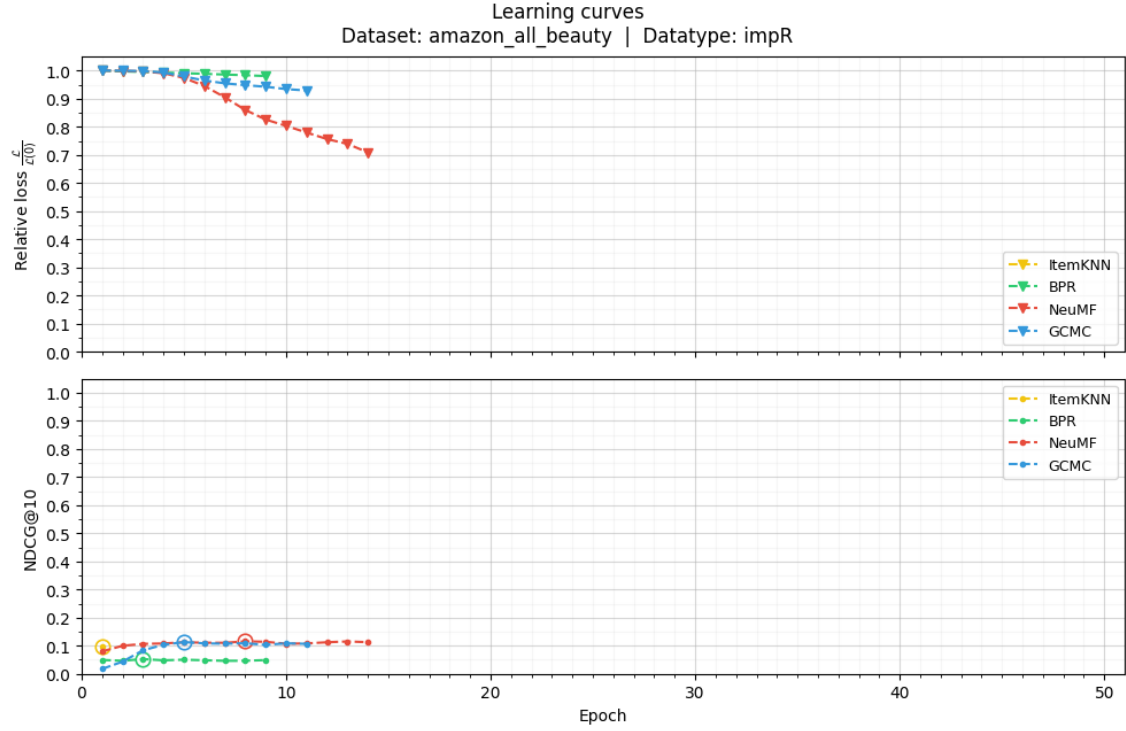


Figure 11: Learning curves for dataset `amazon_all_beauty` with datatype `impR`, showing relative training loss ($\blacktriangledown$) and the early-stopping validation metric NDCG@10 ($\bullet$) over epochs, with highest value ($\odot$) indication. Colors denote models.

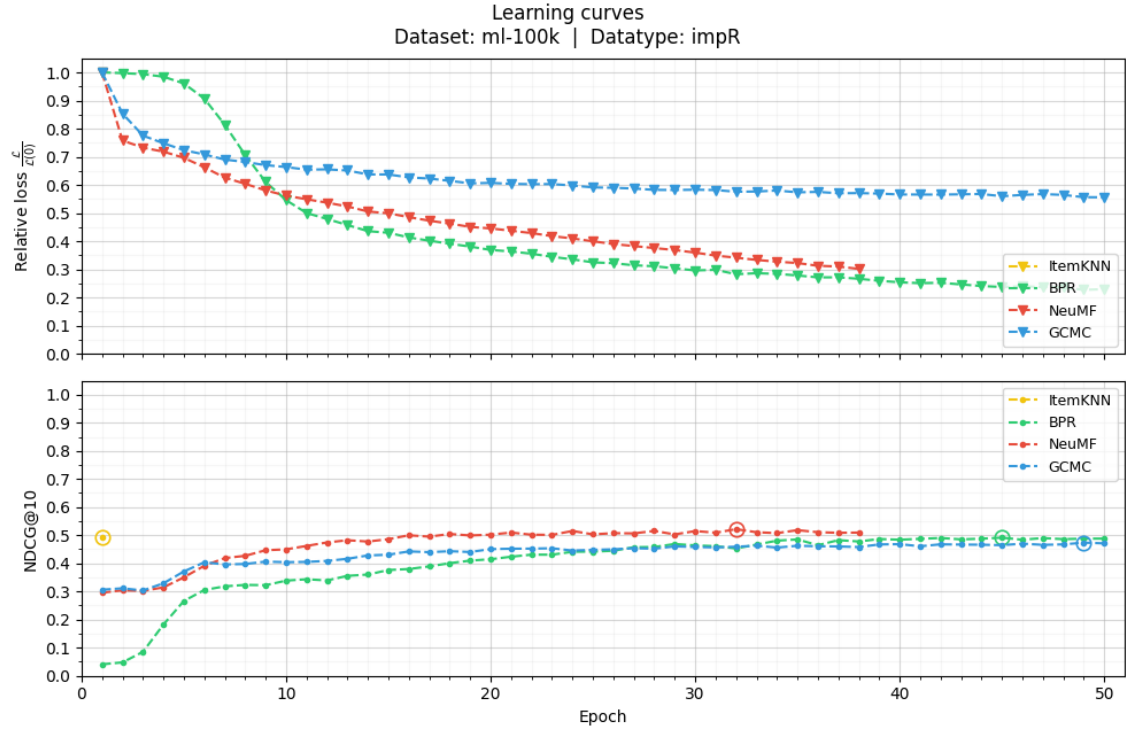


Figure 12: Learning curves for dataset `ml-100k` with datatype `impR`, showing relative training loss ($\blacktriangledown$) and the early-stopping validation metric NDCG@10 ($\bullet$) over epochs, with highest value ($\odot$) indication. Colors denote models.

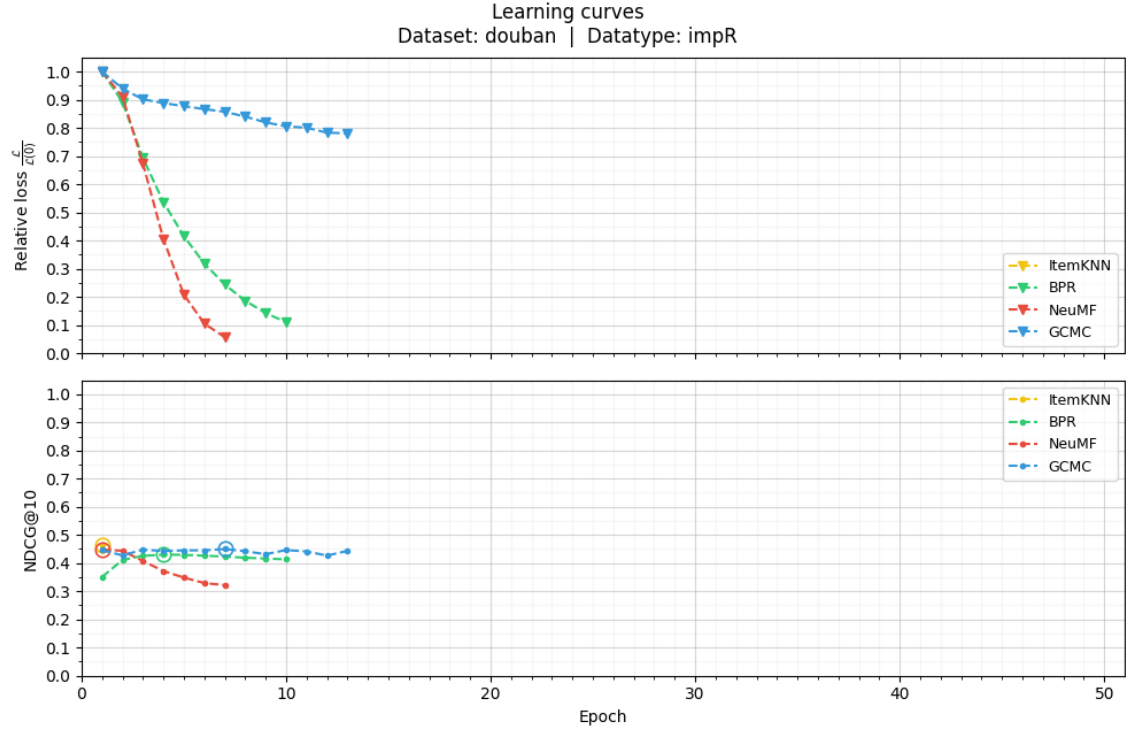


Figure 13: Learning curves for dataset `douban` with datatype `impR`, showing relative training loss ($\blacktriangledown$) and the early-stopping validation metric NDCG@10 ($\bullet$) over epochs, with highest value ($\odot$) indication. Colors denote models.

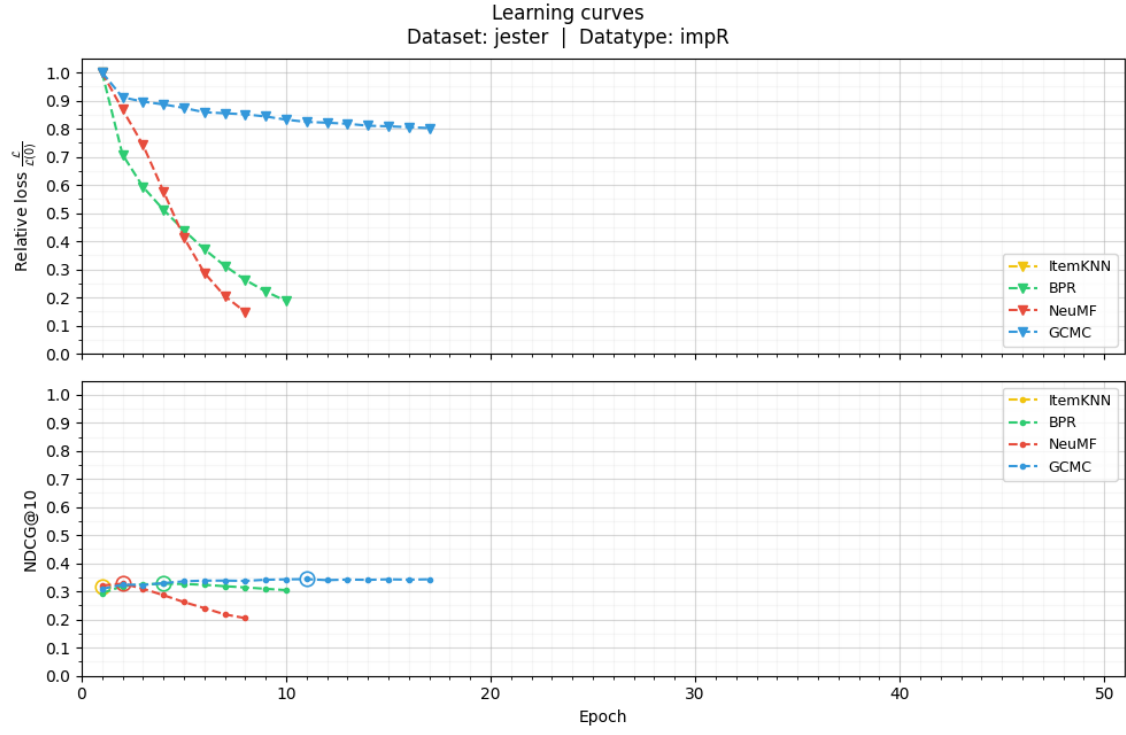


Figure 14: Learning curves for dataset `jester` with datatype `impR`, showing relative training loss ($\blacktriangledown$) and the early-stopping validation metric NDCG@10 ($\bullet$) over epochs, with highest value ($\odot$) indication. Colors denote models.

C Python scripts

This section presents all scripts used in this thesis. All code was implemented and executed in Python notebooks, in preparation for use of Google Colab.

C.1 Data processing

Listing 2: Data_processing.ipynb

```
1 import zipfile
2 import shutil
3 from pathlib import Path
4 import pandas as pd
5
6 def unzip(zip_path: str):
7     """Unzip a specific file, print 'Unzipped', then delete the extracted
8         files."""
9     zip_path = Path(zip_path)
10    if not zip_path.exists() or zip_path.suffix != '.zip':
11        print(f"File not found or not a .zip: {zip_path}")
12        return
13
14    # Extraction folder (same name as zip)
15    extract_dir = zip_path.parent / zip_path.stem
16
17    # Unzip the file
18    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
19        zip_ref.extractall(extract_dir)
20    print("extract_dir Unzipped")
21
22 def cleanup(folder_path: str):
23     folder_path = Path(folder_path)
24     if folder_path.exists():
25         shutil.rmtree(folder_path)
26         print(f"Deleted extracted folder: {folder_path}")
27
28 def filter_users(df_inter: pd.DataFrame, item_cutoff: float):
29     df = df_inter.copy()
30     duplicates = df.duplicated(subset=['user_id:token', 'item_id:token'],
31                               keep=False)
32     if duplicates.any():
33         print("duplicate found")
34         df = (df.sort_values('rating:float', ascending=False)
35               .drop_duplicates(subset=['user_id:token', 'item_id:token'],
36                               keep='first'))
37
38     # Remove only the cutoff users entirely
39     pr_user_item_counts =
40         df_inter.groupby('user_id:token')['item_id:token'].nunique()
41     item_cutoff_users = pr_user_item_counts[pr_user_item_counts <
42         item_cutoff].index
43     df = df[~df['user_id:token'].isin(item_cutoff_users)]
44
45     # For users who interacted with all items, drop ONE random item interaction
46     unique_items_count = df['item_id:token'].nunique()
47     all_item_users = pr_user_item_counts[pr_user_item_counts >=
48         unique_items_count].index
49     rows_to_drop = (
50         df[df['user_id:token'].isin(all_item_users)]
```

```

46         .groupby('user_id:token')
47         .sample(1)
48     )
49     df = df.drop(rows_to_drop.index)
50
51     return df
52
53 def sparsity(df_inter: pd.DataFrame):
54     item_count = df_inter['item_id:token'].nunique()
55     user_count = df_inter['user_id:token'].nunique()
56     interaction_count = len(df_inter)
57
58     print("Sparsity = "
59           f"(({user_count}*{item_count})-{interaction_count})/"
60           f"({user_count}*{item_count}) = "
61           f"{((user_count*item_count)-interaction_count)/(user_count*item_count)}")
62
63 def split_train_valid_test(df: pd.DataFrame, seed: int = 22):
64     df = df.copy()
65
66     # ensure random but reproducible sampling
67     # 1 validation interaction per user
68     val_idx = (
69         df.groupby('user_id:token', group_keys=False)
70         .sample(n=1, random_state=seed)
71         .index
72     )
73     remaining = df.drop(index=val_idx)
74
75     # 1 test interaction per user from remaining
76     test_idx = (
77         remaining.groupby('user_id:token', group_keys=False)
78         .sample(n=1, random_state=seed + 1)
79         .index
80     )
81
82     df_val = df.loc[val_idx].copy()
83     df_test = df.loc[test_idx].copy()
84     df_train = df.drop(index=val_idx.union(test_idx)).copy()
85
86     return df_train, df_val, df_test
87
88
89 zip_path = "./data/zip/"
90 save_path = "./data/split/"
91 datasets = ['amazon_all_beauty', 'ml-100k', 'douban', 'jester']
92 datatypes = ['expR', 'impR', 'impB']
93 rating = [(1,5), (1,5), (1,5), (-10,10)]
94
95 Amazon, MovieLens, Douban, Jester = range(4)
96
97 item_cutoff = 5 # <-- Make changes here
98
99 for d in [MovieLens, Jester, Amazon, Douban]: # <-- Datasets here
100
101     r1, ru = rating[d]
102     good_rating = (r1 + ((ru-r1)*0.75))
103     dataset = datasets[d]
104     #-----
105

```

```

106 unzip(f"{zip_path}{dataset}.zip")
107
108 # --- Load the .inter file ---
109 df_inter = pd.read_csv(f"{zip_path}{dataset}/{dataset}.inter", sep="\t")
110
111 # --- Keep only desired collumms ---
112 desired_cols = ['user_id:token', 'item_id:token', 'rating:float']
113 df_inter = df_inter[[c for c in desired_cols if c in df_inter.columns]]
114 print(".inter file opened")
115 sparsity(df_inter)
116
117 # --- Filtering users for explicit rating ---
118 df_inter_exp = filter_users(df_inter, item_cuttoff)
119 print("users filtered - for explicit")
120 sparsity(df_inter_exp)
121
122 df_inter_imp = filter_users(df_inter_exp[df_inter_exp['rating:float'] >=
    good_rating].copy(),
123                             item_cuttoff)
124 print("users filtered - for emplicit")
125 sparsity(df_inter_imp)
126
127 exp_train, exp_valid, exp_test = split_train_valid_test(df_inter_exp)
128 imp_train, imp_valid, imp_test = split_train_valid_test(df_inter_imp)
129 print("splits created (train/valid/test)")
130
131 # --- save files ---
132
133 # explicit rating interaction files
134 base_path = f"{save_path}{dataset}/{dataset}--expR/{dataset}--expR"
135 Path(base_path).parent.mkdir(parents=True, exist_ok=True)
136 exp_train.to_csv(f"{base_path}.train.inter", sep='\t', index=False)
137 exp_valid.to_csv(f"{base_path}.valid.inter", sep='\t', index=False)
138 exp_test.to_csv(f"{base_path}.test.inter", sep='\t', index=False)
139
140 # implicit rating interaction files
141 base_path = f"{save_path}{dataset}/{dataset}--impR/{dataset}--impR"
142 Path(base_path).parent.mkdir(parents=True, exist_ok=True)
143 imp_train.to_csv(f"{base_path}.train.inter", sep='\t', index=False)
144 imp_valid.to_csv(f"{base_path}.valid.inter", sep='\t', index=False)
145 imp_test.to_csv(f"{base_path}.test.inter", sep='\t', index=False)
146
147 # three implicit binary interaction files
148 imp_train.loc[:, 'rating:float'] = 1
149 imp_valid.loc[:, 'rating:float'] = 1
150 imp_test.loc[:, 'rating:float'] = 1
151 # imp_train.rename(columns={'rating:float': 'label:float'}, inplace=True)
152 # imp_valid.rename(columns={'rating:float': 'label:float'}, inplace=True)
153 # imp_test.rename(columns={'rating:float': 'label:float'}, inplace=True)
154
155 base_path = f"{save_path}{dataset}/{dataset}--impB/{dataset}--impB"
156 Path(base_path).parent.mkdir(parents=True, exist_ok=True)
157 imp_train.to_csv(f"{base_path}.train.inter", sep='\t', index=False)
158 imp_valid.to_csv(f"{base_path}.valid.inter", sep='\t', index=False)
159 imp_test.to_csv(f"{base_path}.test.inter", sep='\t', index=False)
160
161
162 print("split .inter files and .item file saved")
163
164 cleanup(f"{zip_path}{dataset}")

```

C.2 Model execution

Listing 3: Model_execution.ipynb

```
1 import numpy as np
2 # Patch for NumPy >= 1.24 where np.bool8 was removed
3 if not hasattr(np, "bool8"):
4     np.bool8 = np.bool_
5 if not hasattr(np, "np.float_"):
6     np.float_ = np.float64
7 if not hasattr(np, "np.complex_"):
8     np.complex_ = np.complex128
9 if not hasattr(np, "np.unicode_"):
10     np.unicode_ = np.str_
11
12 # Patch for printing to log from .ipynb
13 import logging
14 def recbole_reset_logs():
15     logging.shutdown()
16     for handler in logging.root.handlers[:]:
17         handler.close()
18         logging.root.removeHandler(handler)
19
20 # Check for GPU availability
21 import torch
22 print("torch:", torch.__version__)
23 print("cuda available:", torch.cuda.is_available())
24 print("cuda version:", torch.version.cuda)
25 print("device count:", torch.cuda.device_count())
26 if torch.cuda.is_available():
27     print("gpu name:", torch.cuda.get_device_name(0))
28
29
30 from recbole.quick_start import run_recbole
31
32 models = ['ItemKNN', 'BPR', 'NeuMF', 'GCMC']
33 datasets = ['ml-100k', 'jester', 'amazon_all_beauty', 'douban', 'netflix']
34 datatypes = ['expR', 'impR', 'impB']
35
36 MovieLens, Jester, Amazon, Douban, Netflix = range(5)
37 ItemKNN, BPR, NeuMF, GCMC = range(4)
38 expR, impR, impB = range(3)
39
40 for m in [ItemKNN, BPR, NeuMF, GCMC]:
41     for d in [Amazon, MovieLens, Douban, Jester]:
42         for t in [impB, impR, expR]:
43             model = models[m]
44             dataset = datasets[d]
45             datatype = datatypes[t]
46
47             config_dict = {
48                 # --- Model + data selection ---
49                 'model': model,
50                 'data_path': f'./data/split/{dataset}',
51                 'dataset': f'{dataset}--{datatype}',
52                 'benchmark_filename': ['train', 'valid', 'test'],
53
54                 # --- Use GPU ---
55                 'use_gpu': True,
56                 'num_workers': 4,
57                 'pin_memory': True,
```

```

58
59     # --- ---
60     'show_progress': False,
61
62     #--- ---
63     'load_col': {'inter': ['user_id', 'item_id', 'rating']},
64     'RATING_FIELD': 'rating',
65
66     'eval_args': {
67         'split': {'RS': [0, 0, 0]}, # Disable RecBole auto-splitting
68         'mode': 'uni100'},
69     'test_args': {'mode': 'full'},
70
71     # --- Evaluation targets ---
72     'topk': [10, 20], # Top-k recommendations to consider
73     'metrics': ['Precision', 'Recall', 'MRR', 'Hit', 'NDCG'],
74
75     # --- Training schedule + early stopping ---
76     'epochs': 50, # Max epochs
77     'eval_step': 1, # Evaluate every epoch
78     'stopping_step': 5, # Stop if no improvement for 5 evals (i.e., 5
79     # epochs)
80     'valid_metric': 'NDCG@10', # Choose ONE metric for early stopping
81     'valid_metric_bigger': True # because higher NDCG is better
82 }
83
84 recbole_reset_logs()
85 run_recbole(config_dict=config_dict)
86 recbole_reset_logs()

```

C.3 Log analysis and plotting

Listing 4: Log_analysis_and_plotting.ipynb

```
1 import re
2 import glob
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6
7 models = ['ItemKNN', 'BPR', 'NeuMF', 'GCMC']
8 datasets = ['ml-100k', 'jester', 'amazon_all_beauty', 'douban', 'netflix']
9 datatypes = ['expR', 'impR', 'impB']
10 MovieLens, Jester, Amazon, Douban, Netflix = range(5)
11 ItemKNN, BPR, NeuMF, GCMC = range(4)
12 expR, impR, impB = range(3)
13
14 metrics = ['Recall@10', 'Precision@10', 'MRR@10', 'Hit@10', 'NDCG@10'] + \
15           ['Recall@20', 'Precision@20', 'MRR@20', 'Hit@20', 'NDCG@20']
16
17 columns = ['Model', 'Dataset', 'Datatype'] + metrics + ['Epochs', 'Train',
18               'Eval', 'Params', 'FLOPs', 'CPU', 'GPU', 'MEM']
19 df = pd.DataFrame(columns=columns)
20
21 read_path = 'data/log/'
22 save_path = 'data/result/'
23
24 MODEL_COLORS = {
25     'ItemKNN': '#f1c40f', # Yellow
26     'BPR': '#2ecc71', # Green
27     'NeuMF': '#e74c3c', # Red
28     'GCMC': '#3498db', # Blue
29 }
30
31 def read_log(log_path):
32     training_data = {} # To store metric values
33     test_data = {} # To store test result summary
34
35     train_time = 0.0 # To store accumulated training time (in seconds)
36     eval_time = 0.0 # To store accumulated eval time (in seconds)
37
38     trainable_params = None # To store "Trainable parameters: xxxxxxxx"
39     flops = None # To store "FLOPs: xxxxxx.xx"
40
41     cpu_usage = None
42     gpu_usage = None
43     mem_usage = None
44
45     with open(log_path, "r") as f:
46         for line in f:
47             # Extracting per-epoch metrics (mrr@10, recall@10, precision@10, etc.)
48             metric_pattern = r"(\w+@\d+)\s*:\s*([\d.]+)"
49             matches = re.findall(metric_pattern, line)
50             for metric, value in matches:
51                 if metric not in training_data:
52                     training_data[metric] = []
53                 training_data[metric].append(float(value))
54
55             # Extract per-epoch Training Loss.
56             # Example: epoch 0 training [time: 282.57s, train loss: 325.2666]
57             m_loss = re.search(r"train loss:\s*([\d.]+)", line)
```



```

57         if m_loss:
58             if 'loss' not in training_data:
59                 training_data["loss"] = []
60                 training_data['loss'].append(float(m_loss.group(1)))
61
62         # Extract Trainable Parameters.
63         # Example: Trainable parameters: 168128
64         params_match = re.search(r"Trainable parameters:\s*([\d]+)", line)
65         if params_match:
66             trainable_params = int(params_match.group(1))
67
68         # Extract FLOPs.
69         # Example: Tue 18 Nov 2025 23:07:57 INFO FLOPs: 128.0
70         flops_match = re.search(r"FLOPs:\s*([\d.]+)", line)
71         if flops_match:
72             flops = float(flops_match.group(1))
73
74         # Accumulate training and evaluation time from epoch logs
75         m_train =
76             re.search(r"epoch\s+(\d+)\s+training\s+\[time:\s*([\d.]+)s", line)
77         if m_train:
78             train_time += float(m_train.group(2))
79
80         m_eval =
81             re.search(r"epoch\s+(\d+)\s+evaluating\s+\[time:\s*([\d.]+)s",
82                 line)
83         if m_eval:
84             eval_time += float(m_eval.group(2))
85
86         # Match summary lines containing test results
87         if "INFO test result:" in line:
88             test_data = {m[0]: float(m[1] or m[2]) for m in
89                 re.findall(r"\s*([\d.]+)",
90                     (?:\np\.\float64\(([\d.]+\)|([\d.]+))\)", line)}
91
92         m_cpu = re.search(r"\s*CPU\s*\s*([\d.]+)\s*%", line)
93         if m_cpu:
94             cpu_usage = float(m_cpu.group(1))
95
96         # GPU: capture used and total separately
97         m_gpu = re.search(r"\s*GPU\s*\s*([\d.]+)\s*G\s*/\s*([\d.]+)\s*G",
98             line)
99         if m_gpu:
100             gpu_usage = round(100.0 * float(m_gpu.group(1)) /
101                 float(m_gpu.group(2)), 2)
102
103         # Memory: capture used and total separately
104         m_mem =
105             re.search(r"\s*Memory\s*\s*([\d.]+)\s*G\s*/\s*([\d.]+)\s*G",
106                 line)
107         if m_mem:
108             mem_usage = round(100.0 * float(m_mem.group(1)) /
109                 float(m_mem.group(2)), 2)
110
111         epochs = len(next(iter(training_data.values()))) # Get the number of epochs
112
113         return training_data, test_data, train_time, eval_time, epochs,
114             trainable_params, flops, cpu_usage, gpu_usage, mem_usage
115
116 def normalize_loss(loss):

```

```

106     # relative loss
107     loss = np.array(loss)
108     return (loss / loss[0])
109
110 def plot_4models(dataset, datatype, models_list, training_data_list, save_path):
111     """
112     Top: Relative training loss
113     Bottom: NDCG@10
114     One color per model (same color in both plots)
115     """
116
117     fig, (ax_loss, ax_ndcg) = plt.subplots(
118         2, 1, figsize=(10, 7), sharex=True
119     )
120
121     for model, training_data in zip(models_list, training_data_list):
122         epochs = range(len(training_data['loss']))
123         color = MODEL_COLORS[model]
124
125         # ----- LOSS (TOP) -----
126         loss_rel = normalize_loss(training_data['loss'])
127         ax_loss.plot(
128             np.array(epochs)+1,
129             loss_rel,
130             marker='v',
131             linestyle='--',
132             color=color,
133             label=model
134         )
135
136         # ----- NDCG (BOTTOM) -----
137         ax_ndcg.plot(
138             np.array(epochs)+1,
139             training_data['ndcg@10'],
140             marker='.',
141             linestyle='--',
142             color=color,
143             label=model
144         )
145
146         # mark best NDCG epoch
147         best_ep = max(epochs, key=lambda i: training_data['ndcg@10'][i])
148         ax_ndcg.plot(
149             np.array(best_ep)+1,
150             training_data['ndcg@10'][best_ep],
151             marker='o',
152             markersize=9,
153             markerfacecolor='none',
154             markeredgewidth=1.2,
155             color=color
156         )
157
158         # ----- Labels & limits -----
159         ax_loss.set_ylabel("Relative loss " +
160             r"$\frac{\mathcal{L}}{\mathcal{L}(0)}$")
161         ax_ndcg.set_ylabel("NDCG@10")
162         ax_ndcg.set_xlabel("Epoch")
163
164         ax_loss.set_ylim(0, 1.05)
165         ax_ndcg.set_ylim(0, 1.05)

```

```

165 ax_ndcg.set_xlim(0, 51)
166
167 # ax_loss.grid(True)
168 # ax_ndcg.grid(True)
169 #####
170 X_major_ticks = np.arange(0, 51, 10)
171 X_minor_ticks = np.arange(0, 51, 1)
172 Y_major_ticks = np.arange(0, 1.05, 0.1)
173 Y_minor_ticks = np.arange(0, 1.05, 0.05)
174
175 ax_loss.set_yticks(Y_major_ticks)
176 ax_loss.set_yticks(Y_minor_ticks, minor=True)
177 ax_ndcg.set_yticks(Y_major_ticks)
178 ax_ndcg.set_yticks(Y_minor_ticks, minor=True)
179 ax_ndcg.set_xticks(X_major_ticks)
180 ax_ndcg.set_xticks(X_minor_ticks, minor=True)
181
182 # different settings for the grids:
183 # ax.grid(which='both')
184 # ax.grid(which='major', alpha=0.2, color='#CCCCCC', linestyle='--')
185 ax_loss.grid(which='minor', alpha=0.1)
186 ax_loss.grid(which='major', alpha=0.5)
187 ax_ndcg.grid(which='minor', alpha=0.1)
188 ax_ndcg.grid(which='major', alpha=0.5)
189 #####
190
191 # ----- Legends (model-only, clean) -----
192 ax_loss.legend(fontsize=9, loc="lower right")#, title="Model")
193 ax_ndcg.legend(fontsize=9, loc="upper right")#, title="Model")
194
195 fig.suptitle(
196     f"Learning curves\nDataset: {dataset} | Datatype: {datatype}",
197     y=0.92
198 )
199
200 fig.tight_layout(rect=[0, 0, 1, 0.95])
201 plt.savefig(f"{save_path}/{dataset}--{datatype}.png", bbox_inches='tight')
202 plt.close(fig)
203
204
205 def plot_all(save_path = 'data/result/', read_path = 'data/log/'):
206     for t in [expR, impR, impB]:
207         for d in [Amazon, MovieLens, Douban, Jester]:
208
209             dataset = datasets[d]
210             datatype = datatypes[t]
211
212             models_list = []
213             training_data_list = []
214
215             for m in [ItemKNN, BPR, NeuMF, GCMC]:
216                 model = models[m]
217                 pattern = read_path + f'{model}-{dataset}--{datatype}*.log'
218                 logs = glob.glob(pattern)
219
220                 if not logs:
221                     continue
222
223                 training_data, *_ = read_log(logs[0])
224                 models_list.append(model)

```

```

225         training_data_list.append(training_data)
226
227     if len(training_data_list) > 0:
228         plot_4models(dataset, datatype, models_list, training_data_list,
229                     save_path+'plots')
230
231 def table_csv(save_path = 'data/result/', read_path = 'data/log/'):
232     df = pd.DataFrame(columns=columns)
233     for t in [expR, impR, impB]:
234         for d in [Amazon, MovieLens ,Douban, Jester]:
235             for m in [ItemKNN, BPR, NeuMF, GCMC]:
236                 model = models[m]
237                 dataset = datasets[d]
238                 datatype = datatypes[t]
239                 # pattern: model--dataset--datatype-???.log
240                 log_read_pattern =
241                     read_path+f'{model}-{dataset}--{datatype}--*.log'
242                 for log in glob.glob(log_read_pattern):
243                     training_data, test_data, train_time, eval_time, epochs,
244                     trainable_params, flops, cpu, gpu, mem = read_log(log)
245
246                     # --- Test scores ---
247                     row = {
248                         'Model': model,
249                         'Dataset': dataset,
250                         'Datatype': datatype,
251                         'Epochs': epochs,
252                         'Train': round(train_time / epochs, 2),
253                         'Eval': round(eval_time / epochs, 2),
254                         'Params': trainable_params,
255                         'FLOPs': flops,
256                         'CPU': cpu,
257                         'GPU': gpu,
258                         'MEM': mem,
259                     }
260                     # fill metrics (NaN if missing)
261                     for x in metrics:
262                         row[x] = test_data.get(x.lower(), float('NaN'))
263
264                     df.loc[len(df)] = row # append row
265     df.to_csv(save_path+'df.csv', sep='\t', index=False)
266
267 plot_all()
268 table_csv()

```

C.4 Make tables

Listing 5: Make_tables.ipynb

```
1 import pandas as pd
2
3 EFFECTIVENESS_10_COLUMNS = ['Recall@10', 'Precision@10', 'MRR@10', 'Hit@10',
4                             'NDCG@10']
5 EFFECTIVENESS_20_COLUMNS = ['Recall@20', 'Precision@20', 'MRR@20', 'Hit@20',
6                             'NDCG@20']
7 EFFICIENCY_COLUMNS = ['Epochs', 'Train', 'Eval', 'Params', 'FLOPs', 'CPU',
8                       'GPU', 'MEM']
9
10 #####
11 def effectiveness_table(data_cols, table_name):
12     # =====
13     # CONFIGURATION
14     # =====
15     ID_COLUMNS = ['Dataset', 'Datatype', 'Model']
16     COLUMNS_TO_USE = ID_COLUMNS + data_cols
17
18     CSV_PATH = "data/result/df.csv"
19     OUTPUT_TEX = f"data/result/{table_name}.tex"
20     CSV_SEPARATOR = "\t"
21
22     MODEL_ORDER = ["ItemKNN", "BPR", "NeuMF", "GCMC"]
23     DATASET_ORDER = ["amazon_all_beauty", "ml-100k", "douban", "jester"]
24     DATATYPE_ORDER = ["expR", "impR", "impB"]
25
26     # Formatting
27     FLOAT_FMT = "{:.4f}"
28     EPS = 1e-12
29
30     # Row rules (data rows only)
31     DOTTED_EVERY = 4
32     DASHED_EVERY = 12
33
34     # =====
35     # HELPERS
36     # =====
37     def escape_underscores(s: str) -> str:
38         return s.replace("_", r"\_")
39
40     def is_wrapped(s: str, cmd: str) -> bool:
41         s = str(s).strip()
42         return s.startswith(f"\\{cmd}{{{") and s.endswith("}")
43
44     def join_ints(xs):
45         return ",".join(str(x) for x in xs)
46
47     # =====
48     # LOAD DATA
49     # =====
50     df = pd.read_csv(CSV_PATH, sep=CSV_SEPARATOR)
51     df = df[COLUMNS_TO_USE].copy()
52
53     metric_cols = [c for c in df.columns if c not in ID_COLUMNS]
54
55     # Make metrics numeric
```

```

55     for c in metric_cols:
56         df[c] = pd.to_numeric(df[c], errors="coerce")
57
58     # Escape underscores in ID columns
59     for c in ID_COLUMNS:
60         df[c] = df[c].astype(str).map(escape_underscores)
61
62     # Escape underscores in headers too
63     escaped_headers = [escape_underscores(c) for c in df.columns]
64
65     # =====
66     # ORDERING
67     # =====
68     MODEL_ORDER = [escape_underscores(x) for x in MODEL_ORDER]
69     DATASET_ORDER = [escape_underscores(x) for x in DATASET_ORDER]
70     DATATYPE_ORDER = [escape_underscores(x) for x in DATATYPE_ORDER]
71
72     df["Model"] = pd.Categorical(df["Model"], categories=MODEL_ORDER,
73                                 ordered=True)
74     df["Dataset"] = pd.Categorical(df["Dataset"], categories=DATASET_ORDER,
75                                   ordered=True)
76     df["Datatype"] = pd.Categorical(df["Datatype"], categories=DATATYPE_ORDER,
77                                    ordered=True)
78
79     # Dataset -> Datatype -> Model
80     df = df.sort_values(["Dataset", "Datatype", "Model"]).reset_index(drop=True)
81
82     # Numeric copy for aggregation logic
83     df_num = df.copy()
84     for c in metric_cols:
85         df_num[c] = pd.to_numeric(df_num[c], errors="coerce")
86
87     # IMPORTANT: convert ID columns to plain strings before injecting LaTeX
88     macros
89     for c in ID_COLUMNS:
90         df[c] = df[c].astype(str)
91
92     # =====
93     # HIGHLIGHT VALUES (ORIGINAL): bold best model per (dataset, datatype) per
94     metric
95     # =====
96     for c in metric_cols:
97         best_model = df_num.groupby(["Dataset", "Datatype"],
98                                     observed=True)[c].transform("max")
99
100         formatted = []
101         for x, bm in zip(df_num[c].tolist(), best_model.tolist()):
102             if pd.isna(x):
103                 formatted.append("")
104                 continue
105
106             val = float(x)
107             s = FLOAT_FMT.format(val)
108
109             is_bold = (not pd.isna(bm)) and abs(val - float(bm)) <= EPS
110             if is_bold:
111                 s = f"\\textbf{{{s}}}"
112
113             formatted.append(s)

```

```

109         df[c] = formatted
110
111     # =====
112     # RANK-BASED SUMMARY HIGHLIGHTS (scale-invariant)
113     # - Bold MODEL name: best per dataset (averaged over datatypes; mean rank
114       over metrics)
115     # - Underline DATATYPE: best per (dataset, model) (mean rank over metrics)
116     # =====
117
118     # --- 1) Bold MODEL name: best model per DATASET, averaged over DATATYPES
119     # Rank MODELS within each (dataset, datatype) for each metric (best = rank 1)
120     model_ranks = (
121         df_num
122         .groupby(["Dataset", "Datatype"], observed=True,
123                 dropna=False)[metric_cols]
124         .rank(ascending=False, method="average")
125     )
126
127     # Mean rank over metrics -> one score per row (dataset, datatype, model).
128     # Lower is better.
129     row_model_score = model_ranks.mean(axis=1)
130
131     # Average over datatypes -> one score per (dataset, model)
132     ds_model_score = row_model_score.groupby([df_num["Dataset"],
133         df_num["Model"]], observed=True).mean()
134
135     # Pick best model per dataset (min score)
136     best_score = ds_model_score.groupby(level=0, observed=True).transform("min")
137     best_models = ds_model_score.loc[(ds_model_score - best_score).abs() <= EPS]
138     best_model_set = set(best_models.index.tolist()) # {(dataset, model), ...}
139
140     # Bold the model name for all rows belonging to the winning (dataset, model)
141     for i in range(len(df)):
142         key = (df_num.at[i, "Dataset"], df_num.at[i, "Model"])
143         if key in best_model_set:
144             m = df.at[i, "Model"]
145             if m and not is_wrapped(m, "textbf"):
146                 df.at[i, "Model"] = f"\\textbf{{{m}}}"
147
148     # --- Underline DATATYPE: best datatype per MODEL (averaged over all
149           DATASETS), rank-based
150
151     # Rank DATATYPES within each (dataset, model) for each metric (best = rank 1)
152     dt_ranks = (
153         df_num
154         .groupby(["Dataset", "Model"], observed=True, dropna=False)[metric_cols]
155         .rank(ascending=False, method="average")
156     )
157
158     # Mean rank over metrics per row (dataset, datatype, model). Lower is better.
159     row_dt_score = dt_ranks.mean(axis=1)
160
161     # Average over datasets -> score per (model, datatype)
162     model_dt_score = row_dt_score.groupby([df_num["Model"], df_num["Datatype"]],
163         observed=True).mean()
164
165     # Pick best datatype per model (min score)
166     best_score = model_dt_score.groupby(level=0, observed=True).transform("min")

```

```

162 best_model_dts = model_dt_score.loc[(model_dt_score - best_score).abs() <=
    EPS]
163 best_model_dt_set = set(best_model_dts.index.tolist()) # {(model, datatype),
    ...}
164
165 # Underline datatype everywhere it appears for that model (across all
    datasets)
166 for i in range(len(df)):
167     key = (df_num.at[i, "Model"], df_num.at[i, "Datatype"])
168     if key in best_model_dt_set:
169         d = df.at[i, "Datatype"]
170         if d and not is_wrapped(d, "underline"):
171             df.at[i, "Datatype"] = f"\\underline{{{d}}}"
172
173
174 # =====
175 # BUILD TABULARRAY (tblr) LATEX
176 # =====
177 n_cols = len(df.columns)
178 n_data = len(df)
179
180 colspec = ("l" * len(ID_COLUMNS)) + ("r" * (n_cols - len(ID_COLUMNS)))
181
182 dotted_hlines, dashed_hlines = [], []
183 for k in range(1, n_data + 1): # data row index (1-based)
184     hline_index = k + 2 # rule after data row k
185     if k % DASHED_EVERY == 0:
186         dashed_hlines.append(hline_index)
187     elif k % DOTTED_EVERY == 0:
188         dotted_hlines.append(hline_index)
189
190 lines_opts = []
191 if dotted_hlines:
192     lines_opts.append(f"hline{{{join_ints(dotted_hlines)}}} = {{{dashed}}}")
193 if dashed_hlines:
194     idxs = join_ints(dashed_hlines)
195     lines_opts.append(f"hline{{{idxs}}} = {{{1}}}{{-}}{{solid}}")
196     lines_opts.append(f"hline{{{idxs}}} = {{{2}}}{{-}}{{solid}}")
197
198 tblr_opts = ",\n ".join([
199     f"colspec={{ {colspec} }}",
200     "row{1}={{font=\\bfseries\\large}}",
201     f"cell{{{1}}}{{1-{{n_cols}}}}={{{{halign=l}}}}",
202     f"cell{{{2-Z}}}{{1-{{len(ID_COLUMNS)}}}}={{{{halign=l}}}}",
203     f"cell{{{2-Z}}}{{{{len(ID_COLUMNS)+1}}-{{n_cols}}}}={{{{halign=r}}}}",
204
205     "hline{1} = {1}{{-}}{solid}",
206     "hline{1} = {2}{{-}}{solid}",
207     "hline{2} = {1}{{-}}{solid}",
208     "hline{2} = {2}{{-}}{solid}",
209     *lines_opts,
210     "hline{Z} = {1}{{-}}{solid}",
211     "hline{Z} = {2}{{-}}{solid}",
212 ])
213
214 header_row = " & ".join(escaped_headers) + r" \\"
215
216 body_rows = []
217 for _, row in df.iterrows():
218     body_rows.append(" & ".join(map(str, row.tolist())) + r" \")

```



```

219
220 latex_lines = []
221 latex_lines.append("% Auto-generated by Python (pandas) + tabularray")
222 latex_lines.append("% Requires in LaTeX preamble:")
223 latex_lines.append("% \\usepackage{tabularray}")
224 latex_lines.append("% \\UseTblrLibrary{booktabs}")
225 latex_lines.append("")
226 latex_lines.append("\\begin{tblr}[]{")
227 latex_lines.append(" " + tblr_opts)
228 latex_lines.append("}")
229 latex_lines.append(header_row)
230 latex_lines.extend(body_rows)
231 latex_lines.append("\\end{tblr}")
232 latex_lines.append("")
233
234 with open(OUTPUT_TEX, "w", encoding="utf-8") as f:
235     f.write("\n".join(latex_lines))
236
237 print(f"Saved LaTeX table to {OUTPUT_TEX}")
238
239 #####
240
241 def efficiency_table(data_cols=EFFICIENCY_COLUMNS, table_name="efficiency"):
242     # =====
243     # CONFIGURATION
244     # =====
245     ID_COLUMNS = ['Dataset', 'Datatype', 'Model']
246     COLUMNS_TO_USE = ID_COLUMNS + data_cols
247
248     CSV_PATH = "data/result/df.csv"
249     OUTPUT_TEX = f"data/result/{table_name}.tex"
250     CSV_SEPARATOR = "\t"
251
252     MODEL_ORDER = ["ItemKNN", "BPR", "NeuMF", "GCMC"]
253     DATASET_ORDER = ["amazon_all_beauty", "ml-100k", "douban", "jester"]
254     DATATYPE_ORDER = ["expR", "impR", "impB"]
255
256     # Column typing (per your description)
257     STRING_COLS = set(ID_COLUMNS + [])
258     INT_COLS = {'Epochs', 'FLOPs', 'Params'}
259     FLOAT_COLS = {'Train', 'Eval', 'CPU', 'GPU', 'MEM'}
260
261     # Row rules (data rows only)
262     DOTTED EVERY = 4
263     DASHED EVERY = 12
264
265     # =====
266     # HELPERS
267     # =====
268     def latex_escape(s: str) -> str:
269         s = str(s)
270         return (s.replace("\\", r"\textbackslash{")
271                 .replace("&", r"\&")
272                 .replace("%", r"\%")
273                 .replace("$", r"\$")
274                 .replace("#", r"\#")
275                 .replace("_", r"\_")
276                 .replace("{", r"\{")
277                 .replace("}", r"\}")
278                 .replace("~", r"\textasciitilde{")

```

```

279         .replace("^", r"\textasciicircum{}"))
280
281 def join_ints(xs):
282     return ",".join(str(x) for x in xs)
283
284 def fmt_cell(col, val):
285     # empty for missing
286     if pd.isna(val):
287         return ""
288     if col in FLOAT_COLS:
289         return f"{float(val):.2f}"
290     if col in INT_COLS:
291         # Int64 can be numpy/pandas scalar; cast safely
292         return str(int(val))
293     # strings (also escape underscores for LaTeX)
294     return latex_escape(val)
295
296 # =====
297 # LOAD DATA
298 # =====
299 df = pd.read_csv(CSV_PATH, sep=CSV_SEPARATOR)
300 df = df[COLUMNS_TO_USE].copy()
301
302 # =====
303 # TYPES (fix)
304 # =====
305 # Strings first (keep as strings; escape underscores later via fmt_cell)
306 for c in STRING_COLS:
307     if c in df.columns:
308         df[c] = df[c].astype("string")
309
310 # Ints
311 for c in INT_COLS:
312     if c in df.columns:
313         df[c] = pd.to_numeric(df[c], errors="coerce").astype("Int64")
314
315 # Floats
316 for c in FLOAT_COLS:
317     if c in df.columns:
318         df[c] = pd.to_numeric(df[c], errors="coerce").astype(float)
319
320 # =====
321 # ORDERING
322 # =====
323 # (ordering uses raw values; escaping happens at render time)
324 df["Model"] = pd.Categorical(df["Model"], categories=MODEL_ORDER,
325                             ordered=True)
326 df["Dataset"] = pd.Categorical(df["Dataset"], categories=DATASET_ORDER,
327                               ordered=True)
328 df["Datatype"] = pd.Categorical(df["Datatype"], categories=DATATYPE_ORDER,
329                                 ordered=True)
330
331 df = df.sort_values(["Dataset", "Datatype", "Model"]).reset_index(drop=True)
332
333 # Escape underscores in headers
334 escaped_headers = [latex_escape(c) for c in df.columns]
335
336 # =====
337 # BUILD TABULARRAY (tblr) LATEX
338 # =====

```

```

336     n_cols = len(df.columns)
337     n_data = len(df)
338
339     colspec = ("l" * len(ID_COLUMNS)) + ("r" * (n_cols - len(ID_COLUMNS)))
340
341     dotted_hlines, dashed_hlines = [], []
342     for k in range(1, n_data + 1): # data row index (1-based)
343         hline_index = k + 2 # rule after data row k
344         if k % DASHED_EVERY == 0:
345             dashed_hlines.append(hline_index)
346         elif k % DOTTED_EVERY == 0:
347             dotted_hlines.append(hline_index)
348
349     lines_opts = []
350     if dotted_hlines:
351         lines_opts.append(f"hline{{{join_ints(dotted_hlines)}}} = {{{dashed}}}")
352     if dashed_hlines:
353         idxs = join_ints(dashed_hlines)
354         lines_opts.append(f"hline{{{idxs}}} = {{{1}}}{{-}} {{{solid}}}")
355         lines_opts.append(f"hline{{{idxs}}} = {{{2}}}{{-}} {{{solid}}}")
356
357     tblr_opts = ",\n ".join([
358         f"colspec={{ {colspec} }}",
359         "row{1}={{font=\bfseries\large}}",
360         f"cell{{{1}}}{{{1-{n_cols}}}}={{ {halign=1} }}",
361         f"cell{{{2-Z}}}{{{1-{len(ID_COLUMNS)}}}}={{ {halign=1} }}",
362         f"cell{{{2-Z}}}{{{len(ID_COLUMNS)+1}-{n_cols}}}}={{ {halign=r} }}",
363         "hline{1} = {1}{{-}} {{{solid}}}",
364         "hline{1} = {2}{{-}} {{{solid}}}",
365         "hline{2} = {1}{{-}} {{{solid}}}",
366         "hline{2} = {2}{{-}} {{{solid}}}",
367         *lines_opts,
368         "hline{Z} = {1}{{-}} {{{solid}}}",
369         "hline{Z} = {2}{{-}} {{{solid}}}",
370     ])
371
372     header_row = " & ".join(escaped_headers) + r" \\"
373
374     body_rows = []
375     cols = list(df.columns)
376     for _, row in df.iterrows():
377         formatted = [fmt_cell(col, row[col]) for col in cols]
378         body_rows.append(" & ".join(formatted) + r" \\"
379
380     latex_lines = []
381     latex_lines.append("% Auto-generated by Python (pandas) + tabularray")
382     latex_lines.append("% Requires in LaTeX preamble:")
383     latex_lines.append("% \\usepackage{tabularray}")
384     latex_lines.append("% \\UseTblrLibrary{booktabs}")
385     latex_lines.append("")
386     latex_lines.append("\\begin{tblr}[]{"
387     latex_lines.append(" " + tblr_opts)
388     latex_lines.append("}")
389     latex_lines.append(header_row)
390     latex_lines.extend(body_rows)
391     latex_lines.append("\\end{tblr}")
392     latex_lines.append("")
393
394     with open(OUTPUT_TEX, "w", encoding="utf-8") as f:
395         f.write("\n".join(latex_lines))

```

```

396
397     print(f"Saved LaTeX table to {OUTPUT_TEX}")
398
399 effectiveness_table(EFFECTIVENESS_10_COLUMNS, "effectiveness@10")
400 effectiveness_table(EFFECTIVENESS_20_COLUMNS, "effectiveness@20")
401 efficiency_table()

```

D Recbole data

Datasets presented to RecBole must follow the correct configuration format (Figure 15). Depending on the recommendation task, different files may be required. For pure collaborative filtering, referred to by RecBole as a general task, only one file is required, named `.inter`. The `.inter` file contains user-item interactions, with either explicit ratings or implicit labels (Figure 15d).

Suffix	Content	Example Format
<code>.inter</code>	User-item interaction	user_id, item_id, rating, timestamp, review
<code>.user</code>	User feature	user_id, age, gender
<code>.item</code>	Item feature	item_id, category
<code>.kg</code>	Triplets in a knowledge graph	head_entity, tail_entity, relation
<code>.link</code>	Item-entity linkage data	entity, item_id
<code>.net</code>	Social graph data	source, target

(a) File types

Tasks	Mandatory atomic files
General	<code>.inter</code>
Context-aware	<code>.inter</code> , <code>.user</code> , <code>.item</code>
Knowledge-aware	<code>.inter</code> , <code>.kg</code> , <code>.link</code>
Sequential	<code>.inter</code>
Social	<code>.inter</code> , <code>.net</code>

(b) Task types

feat_type	Explanations	Examples
token	single discrete feature	user_id, age
token_seq	discrete features sequence	review
float	single continuous feature	rating, timestamp
float_seq	continuous feature sequence	vector

(c) Data types

user_id:token	item_id:token	rating:float	timestamp:float
1	1193	5	978300760
1	661	3	978302109

(d) `.inter` example

Figure 15: Recbole Atomic files

E Log examples with hyperparameters

This section presents log files for each model, documenting the standard hyperparameters used. Detailed training, evaluation, and test logs have been omitted to conserve space.

E.1 ItemKNN log

Listing 6: ItemKNN, ml-100k, expR

```
1 Mon 29 Dec 2025 13:17:05 INFO ['c:\\Users\\marti\\miniconda3\\envs\\bp_cuda\\Lib
  \\site-packages\\ipykernel_launcher.py', '--f=c:\\Users\\marti\\AppData\\
  Roaming\\jupyter\\runtime\\kernel-v351daa346f84dc3599eb8e4c3c05f129cb23e28a2.
  json']
2 Mon 29 Dec 2025 13:17:05 INFO
3 General Hyper Parameters:
4 gpu_id = 0
5 use_gpu = True
6 seed = 2020
7 state = INFO
8 reproducibility = True
9 data_path = ./data/split/ml-100k\\ml-100k--expR
10 checkpoint_dir = saved
11 show_progress = False
12 save_dataset = False
13 dataset_save_path = None
14 save_data loaders = False
15 data loaders_save_path = None
16 log_wandb = False
17
18 Training Hyper Parameters:
19 epochs = 50
20 train_batch_size = 2048
21 learner = adam
22 learning_rate = 0.001
23 train_neg_sample_args = {'distribution': 'uniform', 'sample_num': 1, 'alpha':
  1.0, 'dynamic': False, 'candidate_num': 0}
24 eval_step = 1
25 stopping_step = 5
26 clip_grad_norm = None
27 weight_decay = 0.0
28 loss_decimal_place = 4
29
30 Evaluation Hyper Parameters:
31 eval_args = {'split': {'RS': [0, 0, 0]}, 'order': 'RO', 'group_by': 'user', 'mode
  ': {'valid': 'uni100', 'test': 'uni100'}}
32 repeatable = False
33 metrics = ['Precision', 'Recall', 'MRR', 'Hit', 'NDCG']
34 topk = [10, 20]
35 valid_metric = NDCG@10
36 valid_metric_bigger = True
37 eval_batch_size = 4096
38 metric_decimal_place = 4
39
40 Dataset Hyper Parameters:
41 field_separator =
42 seq_separator =
43 USER_ID_FIELD = user_id
44 ITEM_ID_FIELD = item_id
45 RATING_FIELD = rating
```

```

46 TIME_FIELD = timestamp
47 seq_len = None
48 LABEL_FIELD = label
49 threshold = None
50 NEG_PREFIX = neg_
51 load_col = {'inter': ['user_id', 'item_id', 'rating']}
52 unload_col = None
53 unused_col = None
54 additional_feat_suffix = None
55 rm_dup_inter = None
56 val_interval = None
57 filter_inter_by_user_or_item = True
58 user_inter_num_interval = [0,inf)
59 item_inter_num_interval = [0,inf)
60 alias_of_user_id = None
61 alias_of_item_id = None
62 alias_of_entity_id = None
63 alias_of_relation_id = None
64 preload_weight = None
65 normalize_field = None
66 normalize_all = None
67 ITEM_LIST_LENGTH_FIELD = item_length
68 LIST_SUFFIX = _list
69 MAX_ITEM_LIST_LENGTH = 50
70 POSITION_FIELD = position_id
71 HEAD_ENTITY_ID_FIELD = head_id
72 TAIL_ENTITY_ID_FIELD = tail_id
73 RELATION_ID_FIELD = relation_id
74 ENTITY_ID_FIELD = entity_id
75 kg_reverse_r = False
76 entity_kg_num_interval = [0,inf)
77 relation_kg_num_interval = [0,inf)
78 benchmark_filename = ['train', 'valid', 'test']
79
80 Other Hyper Parameters:
81 worker = 0
82 wandb_project = recbole
83 shuffle = True
84 require_pow = False
85 enable_amp = False
86 enable_scaler = False
87 transform = None
88 k = 100
89 shrink = 0.0
90 knn_method = item
91 numerical_features = []
92 discretization = None
93 MODEL_TYPE = ModelType.TRADITIONAL
94 num_workers = 4
95 pin_memory = True
96 test_args = {'mode': 'full'}
97 MODEL_INPUT_TYPE = InputType.POINTWISE
98 eval_type = EvaluatorType.RANKING
99 single_spec = True
100 local_rank = 0
101 device = cuda
102 valid_neg_sample_args = {'distribution': 'uniform', 'sample_num': 100}
103 test_neg_sample_args = {'distribution': 'uniform', 'sample_num': 100}
104
105

```

```

106 Mon 29 Dec 2025 13:17:06 INFO ml-100k--expR
107 The number of users: 944
108 Average actions of users: 106.04453870625663
109 The number of items: 1683
110 Average actions of items: 59.45303210463734
111 The number of inters: 100000
112 The sparsity of the dataset: 93.70575143257098%
113 Remain Fields: ['user_id', 'item_id', 'rating']
114 Mon 29 Dec 2025 13:17:06 INFO [Training]: train_batch_size = [2048]
      train_neg_sample_args: [{'distribution': 'uniform', 'sample_num': 1, 'alpha':
      1.0, 'dynamic': False, 'candidate_num': 0}]
115 Mon 29 Dec 2025 13:17:06 INFO [Evaluation]: eval_batch_size = [4096] eval_args:
      [{'split': {'RS': [0, 0, 0]}, 'order': 'RO', 'group_by': 'user', 'mode': {'
      valid': 'uni100', 'test': 'uni100'}}]
116 Mon 29 Dec 2025 13:17:07 INFO ItemKNN()
117 Trainable parameters: 1
118 Mon 29 Dec 2025 13:17:07 INFO FLOPs: 0.0

```

E.2 BPR log

Listing 7: BPR, jester, impR

```

1 Sun 14 Dec 2025 00:18:03 INFO ['c:\\Users\\marti\\miniconda3\\envs\\bp_cuda\\Lib
  \\site-packages\\ipykernel_launcher.py', '--f=c:\\Users\\marti\\AppData\\
  Roaming\\jupyter\\runtime\\kernel-v3c445ff7fe94ae4af53429f549af66ad1608410b0.
  json']
2 Sun 14 Dec 2025 00:18:03 INFO
3 General Hyper Parameters:
4 gpu_id = 0
5 use_gpu = True
6 seed = 2020
7 state = INFO
8 reproducibility = True
9 data_path = ./data/split/jester\\jester--impR
10 checkpoint_dir = saved
11 show_progress = False
12 save_dataset = False
13 dataset_save_path = None
14 save_data loaders = False
15 data loaders_save_path = None
16 log_wandb = False
17
18 Training Hyper Parameters:
19 epochs = 50
20 train_batch_size = 2048
21 learner = adam
22 learning_rate = 0.001
23 train_neg_sample_args = {'distribution': 'uniform', 'sample_num': 1, 'alpha':
      1.0, 'dynamic': False, 'candidate_num': 0}
24 eval_step = 1
25 stopping_step = 5
26 clip_grad_norm = None
27 weight_decay = 0.0
28 loss_decimal_place = 4
29
30 Evaluation Hyper Parameters:
31 eval_args = {'split': {'RS': [0, 0, 0]}, 'order': 'RO', 'group_by': 'user', 'mode
      ': {'valid': 'uni100', 'test': 'uni100'}}
32 repeatable = False
33 metrics = ['Precision', 'Recall', 'MRR', 'Hit', 'NDCG']

```

```

34 topk = [10, 20]
35 valid_metric = NDCG@10
36 valid_metric_bigger = True
37 eval_batch_size = 4096
38 metric_decimal_place = 4
39
40 Dataset Hyper Parameters:
41 field_separator =
42 seq_separator =
43 USER_ID_FIELD = user_id
44 ITEM_ID_FIELD = item_id
45 RATING_FIELD = rating
46 TIME_FIELD = timestamp
47 seq_len = None
48 LABEL_FIELD = label
49 threshold = None
50 NEG_PREFIX = neg_
51 load_col = {'inter': ['user_id', 'item_id', 'rating']}
52 unload_col = None
53 unused_col = None
54 additional_feat_suffix = None
55 rm_dup_inter = None
56 val_interval = None
57 filter_inter_by_user_or_item = True
58 user_inter_num_interval = [0,inf)
59 item_inter_num_interval = [0,inf)
60 alias_of_user_id = None
61 alias_of_item_id = None
62 alias_of_entity_id = None
63 alias_of_relation_id = None
64 preload_weight = None
65 normalize_field = None
66 normalize_all = None
67 ITEM_LIST_LENGTH_FIELD = item_length
68 LIST_SUFFIX = _list
69 MAX_ITEM_LIST_LENGTH = 50
70 POSITION_FIELD = position_id
71 HEAD_ENTITY_ID_FIELD = head_id
72 TAIL_ENTITY_ID_FIELD = tail_id
73 RELATION_ID_FIELD = relation_id
74 ENTITY_ID_FIELD = entity_id
75 benchmark_filename = ['train', 'valid', 'test']
76
77 Other Hyper Parameters:
78 worker = 0
79 wandb_project = recbole
80 shuffle = True
81 require_pow = False
82 enable_amp = False
83 enable_scaler = False
84 transform = None
85 embedding_size = 64
86 numerical_features = []
87 discretization = None
88 kg_reverse_r = False
89 entity_kg_num_interval = [0,inf)
90 relation_kg_num_interval = [0,inf)
91 MODEL_TYPE = ModelType.GENERAL
92 num_workers = 4
93 pin_memory = True

```



```

94 test_args = {'mode': 'full'}
95 MODEL_INPUT_TYPE = InputType.PAIRWISE
96 eval_type = EvaluatorType.RANKING
97 single_spec = True
98 local_rank = 0
99 device = cuda
100 valid_neg_sample_args = {'distribution': 'uniform', 'sample_num': 100}
101 test_neg_sample_args = {'distribution': 'uniform', 'sample_num': 100}
102
103
104 Sun 14 Dec 2025 00:18:07 INFO jester--impR
105 The number of users: 50098
106 Average actions of users: 20.228696329121505
107 The number of items: 101
108 Average actions of items: 10133.97
109 The number of inters: 1013397
110 The sparsity of the dataset: 79.9719875776152%
111 Remain Fields: ['user_id', 'item_id', 'rating']
112 Sun 14 Dec 2025 00:18:08 INFO [Training]: train_batch_size = [2048]
      train_neg_sample_args: [{'distribution': 'uniform', 'sample_num': 1, 'alpha':
      1.0, 'dynamic': False, 'candidate_num': 0}]
113 Sun 14 Dec 2025 00:18:08 INFO [Evaluation]: eval_batch_size = [4096] eval_args:
      [{'split': {'RS': [0, 0, 0]}, 'order': 'RO', 'group_by': 'user', 'mode': {'
      valid': 'uni100', 'test': 'uni100'}}]
114 Sun 14 Dec 2025 00:18:08 INFO BPR(
115   (user_embedding): Embedding(50098, 64)
116   (item_embedding): Embedding(101, 64)
117   (loss): BPRLoss()
118 )
119 Trainable parameters: 3212736
120 Sun 14 Dec 2025 00:18:08 INFO FLOPs: 128.0

```

E.3 NeuMF log

Listing 8: NeuMF, amazon_all_beauty, impB

```

1 Sun 14 Dec 2025 00:57:44 INFO ['c:\\Users\\marti\\miniconda3\\envs\\bp_cuda\\Lib
  \\site-packages\\ipykernel_launcher.py', '--f=c:\\Users\\marti\\AppData\\
  Roaming\\jupyter\\runtime\\kernel-v3c445ff7fe94ae4af53429f549af66ad1608410b0.
  json']
2 Sun 14 Dec 2025 00:57:44 INFO
3 General Hyper Parameters:
4 gpu_id = 0
5 use_gpu = True
6 seed = 2020
7 state = INFO
8 reproducibility = True
9 data_path = ./data/split/amazon_all_beauty\\amazon_all_beauty--impB
10 checkpoint_dir = saved
11 show_progress = False
12 save_dataset = False
13 dataset_save_path = None
14 save_dataloaders = False
15 dataloaders_save_path = None
16 log_wandb = False
17
18 Training Hyper Parameters:
19 epochs = 50
20 train_batch_size = 2048
21 learner = adam

```

```

22 learning_rate = 0.001
23 train_neg_sample_args = {'distribution': 'uniform', 'sample_num': 1, 'alpha':
    1.0, 'dynamic': False, 'candidate_num': 0}
24 eval_step = 1
25 stopping_step = 5
26 clip_grad_norm = None
27 weight_decay = 0.0
28 loss_decimal_place = 4
29
30 Evaluation Hyper Parameters:
31 eval_args = {'split': {'RS': [0, 0, 0]}, 'order': 'RO', 'group_by': 'user', 'mode
    ': {'valid': 'uni100', 'test': 'uni100'}}
32 repeatable = False
33 metrics = ['Precision', 'Recall', 'MRR', 'Hit', 'NDCG']
34 topk = [10, 20]
35 valid_metric = NDCG@10
36 valid_metric_bigger = True
37 eval_batch_size = 4096
38 metric_decimal_place = 4
39
40 Dataset Hyper Parameters:
41 field_separator =
42 seq_separator =
43 USER_ID_FIELD = user_id
44 ITEM_ID_FIELD = item_id
45 RATING_FIELD = rating
46 TIME_FIELD = timestamp
47 seq_len = None
48 LABEL_FIELD = label
49 threshold = None
50 NEG_PREFIX = neg_
51 load_col = {'inter': ['user_id', 'item_id', 'rating']}
52 unload_col = None
53 unused_col = None
54 additional_feat_suffix = None
55 rm_dup_inter = None
56 val_interval = None
57 filter_inter_by_user_or_item = True
58 user_inter_num_interval = [0,inf)
59 item_inter_num_interval = [0,inf)
60 alias_of_user_id = None
61 alias_of_item_id = None
62 alias_of_entity_id = None
63 alias_of_relation_id = None
64 preload_weight = None
65 normalize_field = None
66 normalize_all = None
67 ITEM_LIST_LENGTH_FIELD = item_length
68 LIST_SUFFIX = _list
69 MAX_ITEM_LIST_LENGTH = 50
70 POSITION_FIELD = position_id
71 HEAD_ENTITY_ID_FIELD = head_id
72 TAIL_ENTITY_ID_FIELD = tail_id
73 RELATION_ID_FIELD = relation_id
74 ENTITY_ID_FIELD = entity_id
75 benchmark_filename = ['train', 'valid', 'test']
76
77 Other Hyper Parameters:
78 worker = 0
79 wandb_project = recbole

```

```

80 shuffle = True
81 require_pow = False
82 enable_amp = False
83 enable_scaler = False
84 transform = None
85 mf_embedding_size = 64
86 mlp_embedding_size = 64
87 mlp_hidden_size = [128, 64]
88 dropout_prob = 0.1
89 mf_train = True
90 mlp_train = True
91 use_pretrain = False
92 mf_pretrain_path = None
93 mlp_pretrain_path = None
94 numerical_features = []
95 discretization = None
96 kg_reverse_r = False
97 entity_kg_num_interval = [0,inf)
98 relation_kg_num_interval = [0,inf)
99 MODEL_TYPE = ModelType.GENERAL
100 num_workers = 4
101 pin_memory = True
102 test_args = {'mode': 'full'}
103 MODEL_INPUT_TYPE = InputType.POINTWISE
104 eval_type = EvaluatorType.RANKING
105 single_spec = True
106 local_rank = 0
107 device = cuda
108 valid_neg_sample_args = {'distribution': 'uniform', 'sample_num': 100}
109 test_neg_sample_args = {'distribution': 'uniform', 'sample_num': 100}
110
111
112 Sun 14 Dec 2025 00:57:46 INFO  amazon_all_beauty--impB
113 The number of users: 999
114 Average actions of users: 9.731462925851703
115 The number of items: 4785
116 Average actions of items: 2.0301003344481607
117 The number of inters: 9712
118 The sparsity of the dataset: 99.79682922211659%
119 Remain Fields: ['user_id', 'item_id']
120 Sun 14 Dec 2025 00:57:46 INFO  [Training]: train_batch_size = [2048]
      train_neg_sample_args: [{'distribution': 'uniform', 'sample_num': 1, 'alpha':
      1.0, 'dynamic': False, 'candidate_num': 0}]
121 Sun 14 Dec 2025 00:57:46 INFO  [Evaluation]: eval_batch_size = [4096] eval_args:
      [{'split': {'RS': [0, 0, 0]}, 'order': 'RO', 'group_by': 'user', 'mode': {'
      valid': 'uni100', 'test': 'uni100'}}]
122 Sun 14 Dec 2025 00:57:46 INFO  NeuMF(
123   (user_mf_embedding): Embedding(999, 64)
124   (item_mf_embedding): Embedding(4785, 64)
125   (user_mlp_embedding): Embedding(999, 64)
126   (item_mlp_embedding): Embedding(4785, 64)
127   (mlp_layers): MLP layers(
128     (mlp_layers): Sequential(
129       (0): Dropout(p=0.1, inplace=False)
130       (1): Linear(in_features=128, out_features=128, bias=True)
131       (2): ReLU()
132       (3): Dropout(p=0.1, inplace=False)
133       (4): Linear(in_features=128, out_features=64, bias=True)
134       (5): ReLU()
135     )

```

```

136 )
137 (predict_layer): Linear(in_features=128, out_features=1, bias=True)
138 (sigmoid): Sigmoid()
139 (loss): BCEWithLogitsLoss()
140 )
141 Trainable parameters: 765249
142 Sun 14 Dec 2025 00:57:46 INFO FLOPs: 24960.0

```

E.4 GCMC log

Listing 9: GCMC, douban, expR

```

1 Sun 14 Dec 2025 02:42:51 INFO ['c:\\Users\\marti\\miniconda3\\envs\\bp_cuda\\Lib
  \\site-packages\\ipykernel_launcher.py', '--f=c:\\Users\\marti\\AppData\\
  Roaming\\jupyter\\runtime\\kernel-v3c445ff7fe94ae4af53429f549af66ad1608410b0.
  json']
2 Sun 14 Dec 2025 02:42:51 INFO
3 General Hyper Parameters:
4 gpu_id = 0
5 use_gpu = True
6 seed = 2020
7 state = INFO
8 reproducibility = True
9 data_path = ./data/split/douban\\douban--expR
10 checkpoint_dir = saved
11 show_progress = False
12 save_dataset = False
13 dataset_save_path = None
14 save_data loaders = False
15 data loaders_save_path = None
16 log_wandb = False
17
18 Training Hyper Parameters:
19 epochs = 50
20 train_batch_size = 2048
21 learner = adam
22 learning_rate = 0.001
23 train_neg_sample_args = {'distribution': 'uniform', 'sample_num': 1, 'alpha':
  1.0, 'dynamic': False, 'candidate_num': 0}
24 eval_step = 1
25 stopping_step = 5
26 clip_grad_norm = None
27 weight_decay = 0.0
28 loss_decimal_place = 4
29
30 Evaluation Hyper Parameters:
31 eval_args = {'split': {'RS': [0, 0, 0]}, 'order': 'R0', 'group_by': 'user', 'mode
  ': {'valid': 'uni100', 'test': 'uni100'}}
32 repeatable = False
33 metrics = ['Precision', 'Recall', 'MRR', 'Hit', 'NDCG']
34 topk = [10, 20]
35 valid_metric = NDCG@10
36 valid_metric_bigger = True
37 eval_batch_size = 4096
38 metric_decimal_place = 4
39
40 Dataset Hyper Parameters:
41 field_separator =
42 seq_separator =
43 USER_ID_FIELD = user_id

```

```

44 ITEM_ID_FIELD = item_id
45 RATING_FIELD = rating
46 TIME_FIELD = timestamp
47 seq_len = None
48 LABEL_FIELD = label
49 threshold = None
50 NEG_PREFIX = neg_
51 load_col = {'inter': ['user_id', 'item_id', 'rating']}
52 unload_col = None
53 unused_col = None
54 additional_feat_suffix = None
55 rm_dup_inter = None
56 val_interval = None
57 filter_inter_by_user_or_item = True
58 user_inter_num_interval = [0,inf)
59 item_inter_num_interval = [0,inf)
60 alias_of_user_id = None
61 alias_of_item_id = None
62 alias_of_entity_id = None
63 alias_of_relation_id = None
64 preload_weight = None
65 normalize_field = None
66 normalize_all = None
67 ITEM_LIST_LENGTH_FIELD = item_length
68 LIST_SUFFIX = _list
69 MAX_ITEM_LIST_LENGTH = 50
70 POSITION_FIELD = position_id
71 HEAD_ENTITY_ID_FIELD = head_id
72 TAIL_ENTITY_ID_FIELD = tail_id
73 RELATION_ID_FIELD = relation_id
74 ENTITY_ID_FIELD = entity_id
75 benchmark_filename = ['train', 'valid', 'test']
76
77 Other Hyper Parameters:
78 worker = 0
79 wandb_project = recbole
80 shuffle = True
81 require_pow = False
82 enable_amp = False
83 enable_scaler = False
84 transform = None
85 accum = stack
86 gcn_output_dim = 500
87 embedding_size = 64
88 dropout_prob = 0.3
89 sparse_feature = True
90 class_num = 2
91 num_basis_functions = 2
92 numerical_features = []
93 discretization = None
94 kg_reverse_r = False
95 entity_kg_num_interval = [0,inf)
96 relation_kg_num_interval = [0,inf)
97 MODEL_TYPE = ModelType.GENERAL
98 num_workers = 4
99 pin_memory = True
100 test_args = {'mode': 'full'}
101 MODEL_INPUT_TYPE = InputType.PAIRWISE
102 eval_type = EvaluatorType.RANKING
103 single_spec = True

```

```

104 local_rank = 0
105 device = cuda
106 valid_neg_sample_args = {'distribution': 'uniform', 'sample_num': 100}
107 test_neg_sample_args = {'distribution': 'uniform', 'sample_num': 100}
108
109
110 Sun 14 Dec 2025 02:42:57 INFO  douban--expR
111 The number of users: 108664
112 Average actions of users: 8.903232931172525
113 The number of items: 29
114 Average actions of items: 34551.857142857145
115 The number of inters: 967452
116 The sparsity of the dataset: 69.29947931872245%
117 Remain Fields: ['user_id', 'item_id', 'rating']
118 Sun 14 Dec 2025 02:42:59 INFO  [Training]: train_batch_size = [2048]
      train_neg_sample_args: [{'distribution': 'uniform', 'sample_num': 1, 'alpha':
      1.0, 'dynamic': False, 'candidate_num': 0}]
119 Sun 14 Dec 2025 02:42:59 INFO  [Evaluation]: eval_batch_size = [4096] eval_args:
      [{'split': {'RS': [0, 0, 0]}, 'order': 'RO', 'group_by': 'user', 'mode': {'
      valid': 'uni100', 'test': 'uni100'}}]]
120 Sun 14 Dec 2025 02:43:01 INFO  GCMC(
121   (GcEncoder): GcEncoder(
122     (dropout): Dropout(p=0.3, inplace=False)
123     (sparse_dropout): SparseDropout()
124     (activate): ReLU()
125     (weights_u): ParameterList(  (0): Parameter containing: [torch.float32 of
      size 108693x500 (cuda:0)])
126     (weights_v): ParameterList(  (0): Parameter containing: [torch.float32 of
      size 108693x500 (cuda:0)])
127     (dense_layer_u): Linear(in_features=500, out_features=64, bias=False)
128     (dense_layer_v): Linear(in_features=500, out_features=64, bias=False)
129   )
130   (BiDecoder): BiDecoder(
131     (dropout): Dropout(p=0.0, inplace=False)
132     (weights): ParameterList(
133       (0): Parameter containing: [torch.float32 of size 64x64 (cuda:0)]
134       (1): Parameter containing: [torch.float32 of size 64x64 (cuda:0)]
135     )
136     (dense_layer): Linear(in_features=2, out_features=2, bias=False)
137   )
138   (loss_function): CrossEntropyLoss()
139 )
140 Trainable parameters: 54386696
141 Sun 14 Dec 2025 02:43:01 INFO  FLOPs: 3478176004.0

```



KU's AI-deklaration

Deklaration for anvendelse af generative AI-værktøjer
☒ Jeg/vi har benyttet generativ AI som hjælpemiddel/værktøj (sæt kryds) ☐ Jeg/vi har IKKE benyttet generativ AI som hjælpemiddel/værktøj (sæt kryds) <i>Hvis brug af generativ AI er tilladt til eksamen, men du ikke har benyttet det i din opgave, skal du blot krydse af, at du ikke har brugt GAI, og behøver ikke at udfylde resten.</i>
Oplist, hvilke GAI-værktøjer der er benyttet, inkl. link til platformen (hvis muligt): [ChatGPT (Free version), https://chatgpt.com/]
Beskriv hvordan generativ AI er anvendt i opgaven: 1) <i>Formål (hvad har du/I brugt værktøjet til)</i> a. Proofreading - Correction of sentence structure and punctuation. b. Code assistance. c. Finding relevant and reliable sources. 2) <i>Arbejdsfase (hvornår i arbejdsprocessen har du/I brugt GAI)</i> a. It has been used continuously throughout the entire process as a sparring partner whenever appropriate. GAI has not been used to generate content from nothing, but solely as an impartial proofreader for detecting and correcting errors. 3) <i>Hvad gjorde du/I med outputtet (herunder også, om du/I har redigeret outputtet og arbejdet videre med det)</i> a. All outputs have been treated as suggestions rather than final results and have been further processed. NB. GAI-genereret indhold brugt som kilde i opgaven kræver korrekt brug af citationstegn og kildehenvisning. Læs retningslinjer fra Københavns Universitetsbibliotek på KUnet